DOE Helper

Design Experiments, Not Guesswork

Martin J. Gallagher

2026

DOE Helper: Design Experiments, Not Guesswork

Copyright © 2026 Martin J. Gallagher

Licensed under the GNU General Public License v3.0 or later.

Published with Lulu.com

This book accompanies the `doehelper` Python package, available on PyPI.
Website: doehelper.com

No part of this publication may be reproduced without the prior written permission of the author, except as permitted under the GPL-3.0 license.

First edition, 2026.

Contents

PART I: INTRODUCTION

What Is DOE Helper?
- Why Design of Experiments
- Who This Book Is For
- How to Read This Book

PART II: USER'S GUIDE

1. Getting Started

2. Core Workflow

3. Working with Configurations

4. Choosing a Design

5. Running Experiments

6. Analyzing Results

7. Optimization

8. Advanced Features

9. Best Practices

10. Troubleshooting

PART III: TECHNICAL REFERENCE

CLI Reference

Configuration Schema

Design Types

Analysis Output

Output Formats

Statistical Formulas

Design Evaluation Metrics

PART IV: DESIGN OF EXPERIMENTS — THEORY

Preface

Chapter 1: Why Design of Experiments?

Chapter 2: Statistical Foundations

Chapter 3: Full Factorial Designs

Chapter 4: Fractional Factorial Designs
Chapter 5: Screening Designs
Chapter 6: Response Surface Designs
Chapter 7: Analyzing Results
Chapter 8: Response Surface Methodology
Chapter 9: Practical Troubleshooting

PART I

Introduction

What This Book Covers and How to Use It

What Is DOE Helper?

DOE Helper is a command-line tool that automates the design, execution, and analysis of scientific experiments using Design of Experiments (DOE) methodology. You describe your factors and responses in a JSON configuration file, and the tool does the rest: it picks the optimal experiment plan, generates executable scripts to run each test, collects the results, and delivers a full statistical analysis — complete with ANOVA tables, Pareto charts, response surface models, and optimization recommendations.

Instead of guessing which settings to try or exhaustively testing every possible combination, DOE Helper uses mathematically optimal designs to extract the maximum information from the minimum number of experimental runs. The result is faster answers, lower costs, and statistically defensible conclusions.

Why Design of Experiments?

Every experiment costs something — time, money, compute cycles, materials. Poorly planned experiments waste these resources and can lead to wrong conclusions. The traditional approach of changing one variable at a time (OVAT) misses interactions between factors and requires far more runs than necessary.

Design of Experiments is a systematic method developed by Ronald Fisher in the 1920s and refined over the past century for industrial, scientific, and engineering applications. It lets you:

- **Test multiple factors simultaneously** — finding interactions that OVAT misses
- **Minimize the number of experimental runs** — a fractional factorial can test 7 factors in just 8 runs
- **Quantify which factors matter** — with p-values and confidence intervals, not gut feeling
- **Find optimal settings** — using response surface models and multi-objective optimization
- **Make defensible decisions** — with statistical power analysis and model diagnostics

Who This Book Is For

This book is written for anyone who runs experiments and wants to do so more efficiently:

- **Engineers** optimizing systems, processes, or software configurations
- **Scientists** planning laboratory or field experiments
- **Data scientists** evaluating model hyperparameters
- **Quality engineers** improving manufacturing processes
- **DevOps and SRE teams** tuning infrastructure performance
- Anyone who has ever wondered: *"Which of these 10 knobs actually matters?"*

Prerequisites: Basic familiarity with mean, variance, and standard deviation. No prior statistics coursework is assumed. The tool handles the math; this book explains how to interpret the results.

How to Read This Book

This book has four parts, each designed to stand alone:

Part I (this section) introduces the tool and sets the stage.

Part II: User's Guide is a hands-on walkthrough of every feature. Start here if you want to get running immediately. It covers the complete workflow from writing your first configuration file through analyzing results and finding optimal settings.

Part III: Technical Reference is the authoritative specification. Every CLI command, every configuration field, every design type algorithm, every formula. Use this when you need the exact syntax or want to understand what the tool is computing under the hood.

Part IV: Theory teaches the statistical foundations of DOE. It covers factorial designs, fractional factorials, screening designs, response surface methodology, ANOVA, and practical troubleshooting. Read this to understand *why* the designs work, not just *how* to use them.

Suggested Reading Paths

New to DOE and the tool: Read Part I, then Part II (User's Guide), then Part IV chapters 1–3 for theory. Come back to the rest as needed.

Know DOE, new to the tool: Skim Part I, then read Part II and keep Part III open as a reference.

Already using the tool: Jump straight to Part III for the reference, or Part IV for deeper statistical understanding.

What's Included

The DOE Helper tool supports:

- **13 design types** — full factorial, fractional factorial, Plackett-Burman, Box-Behnken, central composite, Latin hypercube, definitive screening, Taguchi, D-optimal, two mixture designs, and linear/log sweeps
- **13 CLI commands** — generate, analyze, optimize, report, info, power, augment, record, status, init, export-worksheet, export-data, and next-batch
- **220+ ready-made templates** — from chemical reactors to Kubernetes tuning to bread baking
- **Complete statistical analysis** — ANOVA, main effects, interactions, Pareto charts, normal probability plots, model diagnostics, response surface modeling, and multi-objective optimization
- **Automated and manual workflows** — generate runner scripts or record results by hand
- **Interactive HTML reports** — self-contained reports with embedded charts that you can share with your team

Everything in this book is based on version 0.1.0 of the `doehelper` package, installable via `pip install doehelper`.

PART II

User's Guide

Hands-On Walkthrough of Every Feature

Getting Started

What is DOE Helper?

DOE Helper is a command-line tool that brings the power of Design of Experiments (DOE) to software engineers, data scientists, researchers, and anyone who needs to systematically optimize a process. Instead of changing one variable at a time (OVAT) or running random guesses, DOE lets you test multiple factors simultaneously in a structured way that reveals not just which factors matter, but how they interact.

> **What problems does it solve?**
> Tuning database configurations, optimizing compiler flags, finding the best ML hyperparameters, improving chemical processes, calibrating manufacturing settings — any scenario where you have multiple knobs to turn and want to find the best combination efficiently.

Installation

```
Terminal

$ pip install doehelper
$ doe --version
```

For detailed system requirements and dependency information, see the Quick Start page.

Your First Experiment: End-to-End Walkthrough

Let's walk through a complete experiment from scratch. We'll optimize a web server's performance by testing three configuration parameters.

Step 1: Create a configuration file

```
Terminal

# Bootstrap a config with doe init
$ doe init --name "Web Server Tuning" --factors 3 --operation full_factorial
```

Edit the generated `config.json` to define your factors and responses:

```
config.json

{
  "metadata": {
    "name": "Web Server Tuning",
    "description": "Optimize Nginx for throughput and latency"
  },
  "factors": [
    {"name": "worker_connections", "levels": ["512", "2048"], "type": "contin
uous"},
    {"name": "keepalive_timeout", "levels": ["15", "75"],   "type": "continuo
us", "unit": "s"},
    {"name": "gzip_level",        "levels": ["1", "6"],     "type": "continuo
us"}
  ],
  "responses": [
    {"name": "throughput", "optimize": "maximize", "unit": "req/s"},
    {"name": "p99_latency", "optimize": "minimize", "unit": "ms"}
  ],
  "runner": {
    "arg_style": "double-dash",
    "result_file": "json"
  },
  "settings": {
    "operation": "full_factorial",
    "test_script": "benchmark.sh",
    "out_directory": "results"
  }
}
```

Step 2: Preview and generate the design

```
Terminal

# Preview the design matrix
$ doe generate --config config.json --dry-run

# Generate the runner script
$ doe generate --config config.json --seed 42

# Check the plan
$ doe info --config config.json
```

Step 3: Run the experiment

```
Terminal

# Execute all runs
$ bash run_experiments.sh

# Check progress
$ doe status --config config.json
```

Step 4: Analyze results

```
Terminal

$ doe analyze --config config.json
```

Step 5: Find the optimum

```
Terminal

$ doe optimize --config config.json --multi
```

Step 6: Generate a shareable report

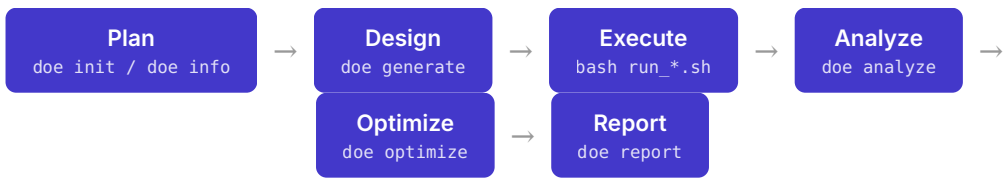
```
Terminal

$ doe report --config config.json --output webserver_report.html
```

✔ **That's the full loop**

In six commands, you went from a question ("which settings matter?") to a statistically rigorous answer with optimization recommendations and a shareable report. The rest of this guide dives deep into each step.

Core Workflow

Every DOE experiment follows the same six-phase lifecycle. The DOE Helper Tool provides commands for each phase.



PHASE	COMMAND(S)	WHAT HAPPENS
Plan	`doe init`, `doe info`, `doe power`	Define factors, responses, and choose a design. Verify the plan makes sense and has sufficient power.
Design	`doe generate`	Create the design matrix and generate a runner script. Use `--dry-run` to preview first.
Execute	`bash run_*.sh`, `doe record`, `doe status`	Run the generated script or manually record results. Track progress with `doe status`.
Analyze	`doe analyze`	Compute ANOVA, effects, generate plots, and identify significant factors.
Optimize	`doe optimize`	Find the best factor settings using response surface methodology.
Report	`doe report`	Generate a self-contained HTML report with all results, plots, and recommendations.

> **Iterative by nature**
> DOE is often iterative. After your first analysis, you may want to augment the design with center or star points, run additional blocks, or switch to a more focused design. The tool supports this workflow with `doe augment` and `doe next-batch`.

Working with Configurations

The configuration file (`config.json`) is the single source of truth for your experiment. Every command reads from it. Understanding its structure is essential.

Anatomy of a Config File

A config file has five top-level sections:

> **metadata**
> Human-readable name and description. Used in reports and status output. Optional but highly recommended.

> **factors**
> The variables you are testing. Each factor has a name, levels (at least 2), and an optional type and unit. This is the heart of your experiment design.

> **responses**
> What you are measuring. Each response has a name, optimization direction (maximize or minimize), and optional unit, weight, and bounds for multi-objective optimization.

> **runner**
> How factors are passed to your test script and how results are collected. Controls argument style and result file format.

> **settings**
> Design type, test script path, output directories, blocking, and design-specific options like LHS sample count.

Factor Types

Choosing the right factor type affects which designs are available and how results are analyzed.

TYPE	WHEN TO USE	EXAMPLES	DESIGN IMPLICATIONS
`continuous`	Numeric values on a scale	Temperature (150–200), Pressure (2–6), Thread count (1–16)	Required for CCD, Box-Behnken, and star points. Enables response surface modeling and interpolation.
`categorical`	Discrete, unordered choices	Algorithm (A, B, C), Cache type (LRU, LFU), Material (steel, aluminum)	Works with all designs. Cannot be interpolated. No star/center points for this factor.
`ordinal`	Ordered categories	Priority (low, medium, high), Compression level (none, fast, best)	Treated as categorical in most designs but ordering is preserved in analysis output.

Response Configuration

```
config.json — responses section

"responses": [
  {
    "name": "throughput",
    "optimize": "maximize",
    "unit": "req/s",
    "weight": 2,           // twice as important in --multi mode
    "bounds": [1000, 5000] // [worst, best] for desirability
  },
  {
    "name": "p99_latency",
    "optimize": "minimize",
    "unit": "ms",
    "weight": 1,
    "bounds": [500, 10]     // [worst, best] — note: worst > best for minimize
  }
]
```

> ⚠ **Bounds direction matters**
> For `maximize` responses, bounds are `[worst, best]` where worst < best. For `minimize` responses, bounds are `[worst, best]` where worst > best. If omitted, bounds are auto-computed from observed data.

Fixed Factors

Fixed factors are held constant across all runs. They are passed to your test script alongside the varying factors but do not contribute to the design matrix.

```
config.json — fixed factors

"fixed_factors": {
  "duration": "60",
  "warmup": "10",
  "num_clients": "100"
}
```

Use fixed factors when a parameter must be specified for your test script to run but you are not currently varying it. They appear in reports for reproducibility.

Runner Configuration

Argument Styles Explained

The `arg_style` setting controls how factor values are passed to your test script:

STYLE	INVOCATION EXAMPLE	BEST FOR
`double-dash`	`./test.sh --temperature 200 --pressure 6 --out run_1.json`	Most scripts. Clear, self-documenting, order-independent.
`env`	`TEMPERATURE=200 PRESSURE=6 OUT=run_1.json ./test.sh`	Docker containers, Makefiles, scripts that read environment variables.
`positional`	`./test.sh 200 6 run_1.json`	Simple scripts. Order must match factor definition order.

Result File Formats

The `result_file` setting controls the expected output format:

json (default) — Your script writes a JSON file with keys matching response names:

```
run_1.json

{"throughput": 3420, "p99_latency": 47.2}
```

Quick Start with Templates

Use `doe init` to generate a starter config file rather than writing one from scratch:

```
Terminal

$ doe init --name "My Experiment" --factors 4 --operation plackett_burman
# Creates config.json with 4 placeholder factors and sensible defaults
```

Choosing a Design

The right design depends on your goals, the number of factors, and how many runs you can afford. This section helps you make that choice.

Decision Guide

> **i**
>
> **Quick rule of thumb**
>
> **Screening** (many factors, limited budget): Use Plackett-Burman or Definitive Screening to find the important factors. **Optimization** (few factors, need the best settings): Use Central Composite or Box-Behnken to model the response surface.

Design Comparison

DESIGN	FACTORS	RUNS (K FACTORS)	DETECTS INTERACTIONS	DETECTS CURVATURE	BEST FOR
Full Factorial	2–5	2^k (e.g., 8, 16, 32)	All	No (2-level)	Complete understanding of a small system
Fractional Factorial	4–8	2^{k-1} or 2^{k-2}	Some (aliased)	No	Moderate screening with fewer runs
Plackett-Burman	4–23	k+1 (next multiple of 4)	No	No	Fast screening of many factors
Definitive Screening	3–12	2k+1	Some	Yes	Modern screening: fewer runs, detects curvature
Central Composite (CCD)	2–6	$2^k + 2k + n_c$	All (2FI)	Yes (quadratic)	Full response surface modeling
Box-Behnken	3–7	Varies (e.g., 15 for 3)	All (2FI)	Yes (quadratic)	RSM without extreme corners
Taguchi	2–15	L4, L8, L12, L16, L27	Limited	No	Robust design, signal-to-noise ratios
Latin Hypercube (LHS)	Any	User-defined	Via analysis	Via analysis	Space-filling, computer experiments
D-Optimal	Any	User-defined	Depends on model	Depends on model	Custom run count, irregular constraints
Mixture (Simplex)	3+	Varies	Via model	Via model	Components that sum to 1 (formulations)

Screening vs. Optimization

DOE typically follows a two-phase approach:

PHASE	GOAL	TYPICAL DESIGNS	RESULT
Phase 1: Screening	Identify the vital few factors from a larger set	Plackett-Burman, Definitive Screening, Fractional Factorial	Narrow from 8–15 factors down to 3–5 important ones
Phase 2: Optimization	Find the best settings for the important factors	CCD, Box-Behnken, Full Factorial	Response surface model with optimal settings

Run Count Reference

Use this table to estimate how many runs your experiment will require:

FACTORS	FULL FACTORIAL (2-LEVEL)	FRACTIONAL (RES IV)	PLACKETT-BURMAN	DEFINITIVE SCREENING	CCD	BOX-BEHNKEN
3	8	—	4	7	20	15
4	16	8	8	9	30	27
5	32	16	8	11	32	46
6	64	16	8	13	52	—
7	128	16	8	15	78	62
8	256	16	12	17	—	—
10	1024	32	12	21	—	—
15	32768	64	16	31	—	—

Design Evaluation Metrics

When you run `doe info --config config.json`, the tool reports three efficiency metrics:

METRIC	WHAT IT MEASURES	GOOD VALUE	PRACTICAL MEANING
D-efficiency	Overall information content of the design	> 90%	Higher means more precise estimates of all effects. An efficiency of 100% means the design is optimal for the given model.
A-efficiency	Average precision of effect estimates	> 80%	Focuses on average variance rather than overall determinant. Good complement to D-efficiency.
G-efficiency	Worst-case prediction variance	> 50%	Ensures no point in the design space has wildly imprecise predictions. Important for RSM.

✔ **Don't overthink it**

For most experiments, the built-in designs are already highly efficient. Use `doe info` to check, and only worry about efficiency if you're using D-Optimal or heavily constrained designs.

Running Experiments

Automated Execution

After `doe generate`, you have a runner script that executes all runs sequentially. The tool supports both Bash and Python formats.

```
Terminal

# Generate a Bash runner (default)
$ doe generate --config config.json --format sh --seed 42

# Execute it
$ bash run_experiments.sh
```

The generated script creates the results directory, runs each combination in randomized order, and saves results as `run_N.json` files.

```
Terminal

# Generate a Python runner
$ doe generate --config config.json --format py --seed 42

# Execute it
$ python run_experiments.py
```

The Python script uses `subprocess` to call your test script. Useful when you need cross-platform compatibility or want to extend the runner logic.

Writing Effective Test Scripts

Your test script is the bridge between the DOE tool and your real experiment. It must follow a simple contract:

1. **Accept factor values** via the configured `arg_style` (double-dash, env, or positional)
2. **Accept** `--out <path>` specifying where to write the result file
3. **Write a JSON file** with keys matching your response names

benchmark.sh — a complete test script example

```
#!/bin/bash
# Parse arguments
while [[ $# -gt 0 ]]; do
  case $1 in
    --worker_connections) WORKERS=$2; shift 2;;
    --keepalive_timeout) KEEPALIVE=$2; shift 2;;
    --gzip_level)        GZIP=$2;     shift 2;;
    --out)               OUT=$2;      shift 2;;
    *) shift;;
  esac
done

# Apply configuration (e.g., update nginx.conf)
sed -i "s/worker_connections .*/worker_connections $WORKERS;/" /etc/nginx/
nginx.conf
sed -i "s/keepalive_timeout .*/keepalive_timeout $KEEPALIVE;/" /etc/nginx/
nginx.conf
sed -i "s/gzip_comp_level .*/gzip_comp_level $GZIP;/" /etc/nginx/nginx.conf
nginx -s reload
sleep 5

# Run the benchmark
RESULT=$(wrk -t4 -c100 -d30s http://localhost:8080/)
THROUGHPUT=$(echo "$RESULT" | grep "Requests/sec" | awk '{print $2}')
LATENCY=$(echo "$RESULT" | grep "99%" | awk '{print $2}' | sed 's/ms//')

# Write results as JSON
echo "{\"throughput\": $THROUGHPUT, \"p99_latency\": $LATENCY}" > "$OUT"
```

Manual Execution with `doe record`

For physical experiments (lab work, field tests, manufacturing trials), you don't need a test script. Use `doe record` to manually enter results:

```
Terminal

# Record a single run
$ doe record --config config.json --run 3

# Record all pending runs interactively
$ doe record --config config.json --run all
```

Tracking Progress

```
Terminal

$ doe status --config config.json

Experiment: Web Server Tuning
Design: full_factorial | 8 runs | 3 factors | 2 responses

Progress: 5/8 complete  [###########........]  63%

Next run to complete: Run 6
  worker_connections = 2048
  keepalive_timeout  = 15 s
  gzip_level         = 6

Record results with: doe record --config config.json --run 6
```

Handling Failures and Recovery

> ⚠ **A run failed or produced bad data?**
> Simply delete the offending `run_N.json` file from the results directory and re-run that specific experiment. The runner script or `doe record` will regenerate it. Use `doe status` to see which runs are missing.

If you cannot complete all runs, use the `--partial` flag on `doe analyze` and `doe optimize` to work with whatever data you have.

Exporting Worksheets

For lab and field work, print a worksheet showing all runs with blank columns for recording results by hand:

```
Terminal

# CSV for Excel or Google Sheets
$ doe export-worksheet --config config.json --format csv --output
worksheet.csv

# Markdown for documentation
$ doe export-worksheet --config config.json --format markdown
```

Analyzing Results

Analysis is where DOE pays off. A single command produces a complete statistical analysis of your experiment.

```
Terminal

$ doe analyze --config config.json
```

What `doe analyze` Produces

- **ANOVA table** — Full analysis of variance with F-tests and p-values
- **Main effects** — How much each factor changes the response
- **Interaction effects** — How factor pairs amplify or cancel each other
- **Confidence intervals** — 95% CI on all effect estimates
- **Summary statistics** — Per-factor, per-level: count, mean, std, min, max
- **Pareto chart** — Visual ranking of factor importance
- **Main effects plot** — Line plots of mean response at each level
- **Normal and half-normal probability plots** — For screening significant effects
- **Model diagnostics** — Residuals, leverage, predicted vs. actual
- **Response surface plots** — 3D surfaces for continuous factor pairs

Reading ANOVA Tables

The ANOVA table is the core output. Here's how to read each column:

COLUMN	FULL NAME	WHAT IT TELLS YOU
Source	Source of Variation	The factor or interaction being tested
SS	Sum of Squares	Total variation explained by this source. Larger = more influence.
DF	Degrees of Freedom	Number of independent comparisons (levels - 1 for main effects)
MS	Mean Square	SS / DF. The average variation per degree of freedom.
F	F-statistic	MS(factor) / MS(error). Large F = the factor has a real effect.
p-value	Probability	Probability that the observed effect is due to chance. **$p < 0.05$ = statistically significant**.

> **i**
>
> **Unreplicated designs**
>
> When you have no replicates (block_count = 1) and a saturated design, there is no independent error term. The tool uses Lenth's method (pseudo-standard error) to estimate significance, the same approach used by R's FrF2 package.

Understanding Main Effects and Interactions

Main effect: The average change in response when a factor moves from its low level to its high level. For example, a main effect of +15 for temperature means increasing temperature raises the response by 15 units on average.

Interaction effect: When the effect of one factor depends on the level of another. If temperature has an effect of +20 when pressure is high but only +5 when pressure is low, there is a temperature-pressure interaction.

Reading Pareto Charts

The Pareto chart ranks all effects (main effects and interactions) from largest to smallest. A cumulative contribution line shows how much of the total variation is explained. Typically, 2–3 factors account for 80% or more of the variation (the Pareto principle).

> ✔ **What to look for**
>
> Focus on the factors above the significance line (dashed red line). These are the "vital few" that truly drive your response. Everything below the line is noise.

Model Diagnostics

The diagnostic panel contains four plots that help you validate the analysis:

PLOT	WHAT TO LOOK FOR	PROBLEM SIGN
Residuals vs. Fitted	Random scatter around zero	Funnel shape (non-constant variance) or curved pattern (model misfit)
Normal Probability (Q-Q)	Points following the diagonal line	Systematic departures indicate non-normal errors
Residuals vs. Run Order	No trends over time	Drift or trending suggests an uncontrolled variable changing during the experiment
Predicted vs. Actual	Points close to the 45-degree line	Systematic deviation means the model is missing important terms

Partial Analysis and Filtering

```
Terminal

# Analyze with incomplete data (some runs not yet finished)
$ doe analyze --config config.json --partial

# Skip generating plots (faster, for CI/CD or headless servers)
$ doe analyze --config config.json --no-plots

# Export results to CSV for further analysis in R, Excel, etc.
$ doe analyze --config config.json --csv exports/
```

Optimization

Once you understand which factors matter, optimization finds the best settings.

Single-Response Optimization

```
Terminal

# Optimize for a specific response
$ doe optimize --config config.json --response throughput
```

The optimizer does three things:

1. **Reports the best observed run** — the actual run with the best measured result
2. **Fits a response surface model** — linear and quadratic models using least squares
3. **Finds the true optimum** — uses L-BFGS-B optimization with multi-start to find the global optimum on the fitted surface, which may lie between tested levels

Response Surface Modeling

Response Surface Methodology (RSM) fits a mathematical model to your data, then uses that model to predict the response at any point in the factor space — even points you didn't test. This is what allows the optimizer to find settings between your tested levels.

> **Linear vs. Quadratic models**
>
> The tool automatically fits both. A **linear model** captures main effects and interactions. A **quadratic model** also captures curvature (the sweet spot). If the quadratic model fits significantly better (higher R-squared), the optimum likely lies in the interior of the factor space rather than at a corner.

Multi-Objective Optimization

When you have conflicting goals (e.g., maximize throughput AND minimize latency), use `--multi` to find the best compromise:

```
Terminal

$ doe optimize --config config.json --multi

============================================================
MULTI-OBJECTIVE OPTIMIZATION
Method: Derringer-Suich Desirability Function
============================================================

Overall desirability: D = 0.7432

Response              Weight Desirability    Predicted  Direction
-----------------------------------------------------------------
throughput               2.0       0.8125     3842 req/s   ↑
p99_latency              1.0       0.6215     38.5 ms      ↓

Recommended settings:
  worker_connections = 1820
  keepalive_timeout  = 62 s
  gzip_level         = 3

Trade-off summary:
  throughput:  3842 (best observed: 4210, sacrifice: +368)
  p99_latency: 38.5 (best observed: 28.1, sacrifice: +10.4)
```

The **desirability function** converts each response into a 0–1 scale (0 = worst, 1 = best), then combines them using a weighted geometric mean. The weights in your config control the relative importance of each response.

Steepest Ascent/Descent

For sequential experimentation (Phase 1 RSM), use `--steepest` to generate a table of follow-up experiments along the gradient direction:

```
Terminal

$ doe optimize --config config.json --response throughput --steepest

Steepest Ascent Path (for throughput, maximize):
Step  worker_connections  keepalive_timeout  gzip_level  Predicted
  0            1280.0              45.0          3.5       3120
  1            1536.0              51.0          3.0       3340
  2            1792.0              57.0          2.5       3560
  3            2048.0              63.0          2.0       3780
  4            2304.0              69.0          1.5       4000

Run these experiments and check if the response keeps improving.
Stop when the response starts declining — you've found the region of the
optimum.
```

> ✔ **When to use steepest ascent**
>
> Use it when your initial screening design suggests the optimum is outside the tested region. Follow the path of improvement until the response starts declining, then run a new RSM design (CCD or Box-Behnken) centered at the new best point.

Advanced Features

Design Augmentation

Augmentation lets you add runs to an existing design without re-running what you've already completed. This is essential for sequential experimentation.

Fold-Over: De-aliasing Confounded Effects

If your fractional factorial design has aliased (confounded) effects and you need to separate them, a fold-over mirrors all factor levels (swaps high and low). This doubles your run count but breaks the aliasing.

```
Terminal

$ doe augment --config config.json --type fold_over
```

When to use: After a fractional factorial shows significant effects, but you can't tell which aliased term is really driving the response.

Star Points: Enabling RSM After Factorial

Adding axial (star) points to a 2-level factorial design converts it into a Central Composite Design, enabling quadratic response surface modeling.

```
Terminal

$ doe augment --config config.json --type star_points
```

When to use: When your factorial analysis reveals significant curvature and you need to model the response surface to find the true optimum.

Center Points: Detecting Curvature

Adds 3 replicate runs at the center of the design space (midpoint of all continuous factors). These runs serve two purposes: detecting curvature and estimating pure error.

```
Terminal

$ doe augment --config config.json --type center_points
```

When to use: After a 2-level design when you suspect the relationship isn't purely linear. If center points show significantly different results from the factorial average, curvature is present.

Power Analysis

Before running an expensive experiment, check whether your design has enough runs to detect the effects you care about:

```
Terminal

$ doe power --config config.json --sigma 5.0 --delta 10.0

Power Analysis
  Design: full_factorial (8 runs)
  Error std dev (sigma): 5.0
  Min detectable effect (delta): 10.0
  Significance level (alpha): 0.05

  Power: 0.93

  Interpretation: You have a 93% chance of detecting an effect
  of size 10.0 or larger. This is above the recommended threshold
  of 0.80. Your design has sufficient power.
```

> ⚠ **Power below 0.80?**
> If power is low, you have several options: (1) increase `block_count` to add replicates, (2) switch to a design with more runs, (3) accept that you can only detect larger effects. Don't run an underpowered experiment — you'll spend time and resources without being able to draw conclusions.

Adaptive/Sequential Experimentation

Use `doe next-batch` for iterative experimentation where each batch of runs informs the next:

```
Terminal

$ doe next-batch --config config.json
```

This command analyzes your current results and suggests the most informative next set of experiments to run, focusing on regions of the factor space where the model is most uncertain.

HTML Report Generation

```
Terminal

$ doe report --config config.json --output experiment_report.html
```

The report is fully self-contained HTML with all plots embedded as base64 images. No external dependencies. Share via email, Slack, wiki, or any file-sharing tool.

Contents include: design summary, ANOVA tables, effects tables, Pareto charts, main effects plots, normal/half-normal probability plots, diagnostic panels, 3D response surfaces, optimization recommendations, and the full design matrix.

Integration with AI Assistants

Use the DOE Helper with AI tools like ChatGPT or Claude for:

- Generating config files from natural language descriptions
- Interpreting analysis output in plain English
- Getting recommendations on which design to use
- Explaining statistical concepts in context

See the AI Prompts page for ready-to-use prompts.

Best Practices

How Many Factors Are Too Many?

FACTORS	APPROACH	RATIONALE
2–5	Full factorial or CCD	Affordable run count. Full interaction analysis.
6–12	Screen first (PB or DSD), then optimize top 3–5	Too many runs for full factorial. Screen to find the vital few.
13+	Plackett-Burman screening, then sequential refinement	Screening is mandatory. Only test the survivors in detail.

Replication and Blocking

> **Why replicate?**
>
> Replication (via `block_count`) gives you an independent estimate of experimental error, makes significance tests more powerful, and helps you detect if conditions changed between batches. For critical experiments, always use at least 2 blocks.

Common Mistakes and How to Avoid Them

Changing one factor at a time (OVAT)

The mistake: Testing factors one at a time while holding others constant.

Why it's bad: OVAT requires more runs than DOE and completely misses interaction effects. Two factors might be fine individually but terrible (or great) together.

The fix: Use any DOE design. Even a simple full factorial with 3 factors and 2 levels (8 runs) gives you complete information about all main effects AND all interactions.

Not randomizing run order

The mistake: Running experiments in a systematic order (all low values first, then all high values).

Why it's bad: If conditions drift over time (temperature, load, degradation), your results will be confounded with the time trend.

The fix: The DOE Helper randomizes run order automatically. Use `--seed` for reproducible randomization.

Testing too many levels

The mistake: Using 5 levels for every factor "to be thorough."

Why it's bad: A 5-factor experiment with 5 levels each = 3125 runs. With 2 levels it's just 32 runs (or 16 with fractional factorial).

The fix: Start with 2 levels per factor. If curvature exists, add center points or switch to CCD. You rarely need more than 3 levels for continuous factors.

Ignoring practical significance

The mistake: Declaring a factor important because $p < 0.05$, even though its actual effect is tiny.

Why it's bad: Statistical significance is not the same as practical significance. A factor that changes throughput by 0.1% may be "significant" with enough data but not worth optimizing.

The fix: Always look at the effect size (the actual magnitude of change), not just the p-value. Use the Pareto chart to see the relative contribution of each factor.

When to Use DOE vs. Other Methods

SITUATION	METHOD	WHY
Multiple factors, want interactions	**DOE**	Designed for this exact problem
Single continuous parameter to tune	Grid search or bisection	DOE is overkill for 1 parameter
Huge search space, cheap evaluations	Bayesian optimization	Better for 20+ dimensional spaces with fast evaluations
Need to understand causal relationships	**DOE**	Only controlled experimentation proves causation
Observational data only	Regression / ML	Can't run experiments, must work with what you have

Tips for Getting the Most Out of Your Experiments

1. **Always preview first.** Use `doe generate --dry-run` and `doe info` before committing.
2. **Check power before running.** Use `doe power` to verify your design can detect the effects you care about.
3. **Start with screening.** If you have more than 5 factors, screen first with PB or DSD before investing in RSM.

4. **Use center points.** They're cheap (3 extra runs) and tell you if curvature exists.
5. **Examine diagnostics.** Always check the residual plots. A bad model gives bad recommendations.
6. **Document everything.** Use `doe report` to generate a permanent record of your experiment.
7. **Iterate.** Your first experiment rarely gives the final answer. Use what you learn to design a better follow-up.

Troubleshooting

Common Error Messages

ERROR	CAUSE	SOLUTION
`No result files found`	The results directory is empty or doesn't exist	Check that `out_directory` in config matches where results are saved. Run `doe status` to verify.
`Factor X requires exactly 2 levels`	Fractional factorial, PB, or CCD requires 2-level factors	Ensure all factors have exactly 2 levels, or switch to a design that supports more.
`Box-Behnken requires 3+ factors`	You have fewer than 3 factors	Use CCD or full factorial for 2-factor experiments instead.
`Response 'X' not found in run_N.json`	The result file is missing a key that matches your response name	Verify that your test script writes JSON with keys exactly matching the `name` fields in your `responses` config.
`Duplicate factor names`	Two factors have the same `name`	Every factor name must be unique. Check for typos.
`Invalid JSON in config file`	Syntax error in config.json	Check for trailing commas, missing quotes, or mismatched brackets. Use a JSON validator.

Missing Results

If some runs failed or weren't completed:

```
Terminal

# See which runs are missing
$ doe status --config config.json

# Analyze with available data only
$ doe analyze --config config.json --partial

# Re-record a specific failed run
$ doe record --config config.json --run 5
```

Design Validation Failures

The tool validates your config against the selected design type. Common validation issues:

- **CCD with categorical factors:** CCD requires all factors to be continuous with exactly 2 levels. Use full factorial for mixed factor types.
- **Definitive Screening with 2 factors:** DSD requires at least 3 factors. Use full factorial instead.
- **Fractional factorial with 3+ levels:** Fractional factorial is only defined for 2-level factors. Use Taguchi for multi-level screening.

Result Format Issues

> ⚠ **Check your JSON output**
>
> The most common issue is response names in the result file not matching the config. Names are case-sensitive. If your config says `"throughput"`, your result file must use `"throughput"` — not `"Throughput"` or `"THROUGHPUT"`.

Verify a result file is valid:

```
Terminal

# Check that the file is valid JSON with the expected keys
$ python -m json.tool results/run_1.json
{
    "throughput": 3420,
    "p99_latency": 47.2
}
```

For additional help, see the Quick Start for command reference, Theory for statistical background, or AI Prompts to get AI-assisted explanations of your results.

PART III

Technical Reference

Complete CLI, Configuration, and Formula Specification

Part 1: CLI Reference

The `doe` command-line interface provides 13 subcommands for generating, running, analyzing, and managing experimental designs. All commands follow the pattern:

```
Terminal

$ doe <command> [options]
```

> **Global Behavior**
>
> The `--config` flag accepts any valid JSON file path. Relative paths are resolved from the current working directory. Most commands that read results use the `out_directory` field from the config unless overridden with `--results-dir`.

doe generate

Generate a design matrix and write a runner script that executes all experimental runs.

FLAG	TYPE	DEFAULT	DESCRIPTION
`--config FILE`	string	*required*	Input JSON configuration file defining factors, responses, and design settings.
`--output PATH`	string	`run_experiments.sh`	Output path for the generated runner script.
`--format {sh,py}`	choice	`sh`	Script format. `sh` produces a Bash script; `py` produces a Python script.
`--seed INT`	integer	*none*	Random seed for run-order randomization. If omitted, run order is not randomized.
`--dry-run`	flag	*off*	Print the design matrix to stdout without writing any files.

```
Example

$ doe generate --config experiment.json --seed 42 --format py
$ doe generate --config experiment.json --dry-run
```

doe analyze

Analyze completed experiment results. Computes effects, ANOVA, generates plots, and produces an HTML report.

FLAG	TYPE	DEFAULT	DESCRIPTION
`--config FILE`	string	*required*	Input JSON configuration file.
`--results-dir DIR`	string	*from config*	Override the `out_directory` setting from the config file.
`--no-plots`	flag	*off*	Skip generating Pareto, main effects, interaction, and diagnostic plots.
`--no-report`	flag	*off*	Skip generating the HTML analysis report.
`--csv DIR`	string	*none*	Export analysis results (effects, ANOVA, summaries) as CSV files into `DIR`.
`--partial`	flag	*off*	Analyze only the runs that have completed so far (ignore missing results).
`--knee`	flag	*off*	Detect saturation/knee points in response curves where diminishing returns begin.
`--factor NAME [NAME...]`	string(s)	*all*	Restrict analysis to the specified factor(s) only.

Example

```
$ doe analyze --config experiment.json --csv exports/
$ doe analyze --config experiment.json --partial --knee --factor temperature
pressure
```

doe info

Display summary information about a design: number of factors, levels, design type, run count, and design evaluation metrics.

FLAG	TYPE	DEFAULT	DESCRIPTION
`--config FILE`	string	*required*	Input JSON configuration file.

doe optimize

Find optimal factor settings from experiment results. Supports single-response optimization, multi-objective desirability, and steepest ascent/descent pathways.

FLAG	TYPE	DEFAULT	DESCRIPTION
`--config FILE`	string	*required*	Input JSON configuration file.
`--results-dir DIR`	string	*from config*	Override the `out_directory` setting.
`--response NAME`	string	*all*	Optimize for a specific response variable. By default, considers all responses.
`--partial`	flag	*off*	Use only completed runs for optimization.
`--multi`	flag	*off*	Multi-objective optimization using desirability functions. Requires `bounds` on responses.
`--steepest`	flag	*off*	Show steepest ascent (for maximize) or descent (for minimize) pathway from the current best point.

doe report

Generate a standalone HTML analysis report without running the full analysis pipeline.

FLAG	TYPE	DEFAULT	DESCRIPTION
`--config FILE`	string	*required*	Input JSON configuration file.
`--results-dir DIR`	string	*from config*	Override the `out_directory` setting.
`--output PATH`	string	`report.html`	Output path for the HTML report file.
`--partial`	flag	*off*	Generate a report using only completed runs.

doe record

Manually record response values for one or all runs. Prompts interactively for each response value.

FLAG	TYPE	DEFAULT	DESCRIPTION
`--config FILE`	string	*required*	Input JSON configuration file.
`--run {N|all}`	string	*required*	Run number (1-based) to record, or `all` to iterate through all pending runs.
`--seed INT`	integer	`42`	Random seed used to determine run order (must match the seed used in `generate`).

doe status

Show experiment progress: how many runs are completed, pending, and failed.

FLAG	TYPE	DEFAULT	DESCRIPTION
`--config FILE`	string	*required*	Input JSON configuration file.
`--seed INT`	integer	`42`	Random seed used to determine run order.

doe power

Perform power analysis to determine whether the design has enough runs to detect a given effect size at the specified significance level.

FLAG	TYPE	DEFAULT	DESCRIPTION
`--config FILE`	string	*required*	Input JSON configuration file.
`--sigma FLOAT`	float	*estimated*	Error standard deviation. If omitted and results exist, estimated from residuals.
`--delta FLOAT`	float	*none*	Minimum detectable effect size (the smallest effect you care about).
`--alpha FLOAT`	float	`0.05`	Significance level (Type I error rate).
`--results-dir DIR`	string	*from config*	Override the `out_directory` setting.
`--partial`	flag	*off*	Use only completed runs when estimating sigma from results.

doe augment

Augment an existing design with additional runs. Use to de-alias effects (fold-over), fit quadratic models (star points), or estimate pure error (center points).

FLAG	TYPE	DEFAULT	DESCRIPTION
`--config FILE`	string	*required*	Input JSON configuration file.
`--type {fold_over,star_points,center_points}`	choice	*required*	Augmentation type. `fold_over` reverses all signs; `star_points` adds axial points; `center_points` adds center-point replicates.
`--output PATH`	string	`run_experiments_augmented.sh`	Output path for the augmented runner script.
`--format {sh,py}`	choice	`sh`	Script format for the augmented runner.
`--seed INT`	integer	*none*	Random seed for run-order randomization of augmented runs.

doe init

Initialize a new experiment from a use-case template. Creates a config file and optionally a test script scaffold.

FLAG	TYPE	DEFAULT	DESCRIPTION
`--template NAME`	string	*none*	Use-case template name (e.g., `web-server`, `database`, `ml-hyperparameter`).
`--list`	flag	*off*	List all available templates with descriptions.
`--output-dir DIR`	string	`.`	Directory where the config file and scaffold will be written.

Example

```
$ doe init --list
$ doe init --template web-server --output-dir my-experiment/
```

doe export-worksheet

Export a blank worksheet showing the design matrix with columns for manually recording responses.

FLAG	TYPE	DEFAULT	DESCRIPTION
`--config FILE`	string	*required*	Input JSON configuration file.
`--format {csv,markdown}`	choice	`csv`	Output format: CSV or Markdown table.
`--output FILE`	string	*stdout*	Output file path. Defaults to printing to stdout.
`--seed INT`	integer	`42`	Random seed for run order.

doe export-data

Export the design matrix and collected response values as a flat data file for use in external analysis tools.

FLAG	TYPE	DEFAULT	DESCRIPTION
`--config FILE`	string	*required*	Input JSON configuration file.
`--format {csv,tsv}`	choice	`csv`	Output format: comma-separated or tab-separated.
`--output FILE`	string	*stdout*	Output file path. Defaults to printing to stdout.
`--seed INT`	integer	`42`	Random seed for run order.
`--partial`	flag	*off*	Include only completed runs (skip missing results).

`doe next-batch`

Generate the next batch of runs for sequential experimentation. Analyzes current results and selects new points based on the chosen strategy.

FLAG	TYPE	DEFAULT	DESCRIPTION
`--config FILE`	string	*required*	Input JSON configuration file.
`--results-dir DIR`	string	*from config*	Override the `out_directory` setting.
`--strategy {refine,explore,balanced}`	choice	*balanced*	`refine` : focus near the best region. `explore` : maximize space coverage. `balanced` : mix of both.
`--batch-size INT`	integer	*auto*	Number of new runs to generate. Auto-selects based on factor count if omitted.
`--output PATH`	string	`run_next_batch.sh`	Output path for the batch runner script.
`--format {sh,py}`	choice	`sh`	Script format for the batch runner.
`--seed INT`	integer	*none*	Random seed for run-order randomization.
`--partial`	flag	*off*	Consider only completed runs when planning the next batch.

Part 2: Configuration Schema

The configuration file (`config.json`) is a single JSON object that defines the entire experiment. Every `doe` command reads this file.

Top-level Fields

FIELD	TYPE	REQUIRED	DESCRIPTION
`metadata`	object	No	Contains `name` (string) and `description` (string) for labeling the experiment.
`factors`	array of Factor	**Yes**	Array of factor definitions. Must contain at least 1 factor (2+ for most design types).
`fixed_factors`	object	No	Key-value pairs passed to every run but not varied. Useful for constants.
`responses`	array of ResponseVar	No	Response variables to collect and analyze. If omitted, all keys in result JSON are treated as responses.
`runner`	RunnerConfig	No	Controls how factor values are passed to the test script.
`settings`	object	No	Design type, blocking, output directories, and other operational settings.

Minimal config.json

```
{
  "metadata": { "name": "My Experiment", "description": "Testing two
factors" },
  "factors": [
    { "name": "temperature", "levels": ["60", "80"], "type": "continuous" },
    { "name": "pressure", "levels": ["1.0", "2.0"], "type": "continuous" }
  ],
  "responses": [
    { "name": "yield", "optimize": "maximize" }
  ],
  "settings": {
    "operation": "full_factorial",
    "test_script": "./run_test.sh",
    "out_directory": "results"
  }
}
```

Factor Object

FIELD	TYPE	DEFAULT	DESCRIPTION
`name`	string	*required*	Unique identifier for the factor. Used as the argument name in runner scripts and as column headers in output.
`levels`	array of strings	*required*	At least 2 values. For continuous factors, these define the low and high (and optionally center) points. Values are always strings in the config.
`type`	string	`"categorical"`	`"categorical"` — unordered discrete values. `"continuous"` — numeric range (enables RSM, CCD, etc.). `"ordinal"` — ordered discrete values (enables trend analysis).
`unit`	string	*none*	Display unit (e.g., `"C"`, `"MB"`, `"ms"`). Shown in reports and plots.
`description`	string	*none*	Human-readable description of the factor.
`dtype`	string	`""` (auto)	Force a data type for level values: `"int"`, `"float"`, or `""` for automatic detection.

⚠

Level Values Are Strings

All level values in the JSON config must be strings, even for numeric factors. The tool parses them based on the `type` and `dtype` fields. Example: `"levels": ["100", "200"]`, not `"levels": [100, 200]`.

ResponseVar Object

FIELD	TYPE	DEFAULT	DESCRIPTION
`name`	string	*required*	Must match a key in the result JSON output by the test script.
`optimize`	string	`"maximize"`	`"maximize"` or `"minimize"`. Determines optimization direction and steepest ascent/descent.
`unit`	string	*none*	Display unit shown in reports and plots.
`description`	string	*none*	Human-readable description.
`weight`	float	`1.0`	Relative importance weight for multi-objective optimization. Higher weight gives this response more influence on the overall desirability.
`bounds`	[float, float]	*none*	Two-element array: `[worst, best]`. Required for desirability-based multi-objective optimization (`--multi`). For maximize: worst < best. For minimize: worst > best.

RunnerConfig

FIELD	TYPE	DEFAULT	DESCRIPTION
`arg_style`	string	`"double-dash"`	How factor values are passed to the test script: `"double-dash"` — `./test.sh --temperature 80 --pressure 2.0` `"env"` — `TEMPERATURE=80 PRESSURE=2.0 ./test.sh` `"positional"` — `./test.sh 80 2.0` (in factor definition order)
`result_file`	string	`"json"`	Expected output format from the test script. Currently `"json"` is the supported format.

Settings

FIELD	TYPE	DEFAULT	DESCRIPTION
`operation`	string	`"full_factorial"`	Design type. One of the 13 supported design types (see Part 3).
`test_script`	string	*none*	Path to the test executable. Invoked for each experimental run.
`block_count`	integer	`1`	Number of blocks (replicates) for the design. Values > 1 replicate the entire design matrix.
`out_directory`	string	`"results"`	Directory where result JSON files are stored, one per run.
`processed_directory`	string	*none*	Directory to move result files after processing. If omitted, results stay in `out_directory`.
`lhs_samples`	integer	`0`	Number of sample points for Latin Hypercube designs. `0` = auto: `max(10, 2 * n_factors)`.
`sweep_points`	integer	`0`	Number of points for linear/log sweep designs. `0` = auto.

Part 3: Design Types

The `operation` field in settings selects the experimental design. Each design type has different requirements, run counts, and estimation capabilities.

full_factorial

Full Factorial Design

PROPERTY	VALUE
Operation string	`"full_factorial"`
Purpose	Evaluate all possible combinations of factor levels. The most thorough design — no aliasing, complete information.
Requirements	1+ factors, 2+ levels each. No restrictions on factor types.
Run count	$L_1 \times L_2 \times \ldots \times L_k$ (product of all level counts)
Algorithm	Cartesian product of all factor levels. Optionally randomized and/or blocked.
Estimates	All main effects, all 2-factor interactions, all higher-order interactions up to k-way.

ℹ Run count grows exponentially: a 2^5 design has 32 runs, but a 3^5 design has 243. Consider fractional or screening designs when k > 4 with 3+ levels.

fractional_factorial

Fractional Factorial Design

PROPERTY	VALUE
Operation string	`"fractional_factorial"`
Purpose	Estimate main effects and some interactions with a fraction of the full factorial runs. Trades resolution for efficiency.
Requirements	2+ factors. Exactly 2 levels per factor.
Run count	2^{k-p}, where p is chosen automatically to achieve Resolution III or higher.
Algorithm	Uses generators to define the fraction. The defining relation determines which effects are aliased (confounded).
Estimates	Main effects (aliased with higher-order interactions at Resolution III). At Resolution IV+, main effects are clear of 2-factor interactions.

plackett_burman

Plackett-Burman Design

PROPERTY	VALUE
Operation string	`"plackett_burman"`
Purpose	Screening: identify which factors have significant main effects with minimal runs. Resolution III.
Requirements	2+ factors. Exactly 2 levels per factor.
Run count	N = smallest multiple of 4 that is ≥ k + 1 (e.g., 12 runs for up to 11 factors).
Algorithm	Constructs a Hadamard-like matrix. Extra columns beyond k factors become dummy factors for error estimation.
Estimates	Main effects only. All 2-factor interactions are partially aliased with main effects.

latin_hypercube

Latin Hypercube Sampling

PROPERTY	VALUE
Operation string	`"latin_hypercube"`
Purpose	Space-filling design for exploring a continuous factor space. Good for computer experiments and metamodeling.
Requirements	1+ continuous factors with exactly 2 levels (defining the range endpoints).
Run count	Configurable via `lhs_samples`. Default: max(10, 2k).
Algorithm	Divides each factor range into n equal strata and places exactly one sample in each stratum per factor. Optimized for maximin distance criterion.
Estimates	No formal aliasing structure. Provides a response surface approximation via regression or interpolation. Not orthogonal.

central_composite

Central Composite Design (CCD)

PROPERTY	VALUE
Operation string	`"central_composite"`
Purpose	Response Surface Methodology (RSM). Fits a full second-order (quadratic) model to find optima.
Requirements	2+ continuous factors with exactly 2 levels.
Run count	$2^k + 2k + n_c$ (factorial points + star points + center points). For k=3: 8 + 6 + 6 = 20.
Algorithm	Combines a factorial core (2^k), axial/star points at distance α along each axis, and center-point replicates. The α value is set to make the design rotatable ($\alpha = 2^{k/4}$).
Estimates	All main effects, all 2-factor interactions, all quadratic (squared) terms. Full second-order model.

box_behnken

Box-Behnken Design

PROPERTY	VALUE
Operation string	`"box_behnken"`
Purpose	RSM design that avoids extreme corner points. Useful when corners are infeasible or dangerous.
Requirements	3+ continuous factors with exactly 2 levels.
Run count	Depends on k. For k=3: 12 + center points. For k=4: 24 + center points. Generally fewer runs than CCD.
Algorithm	Combines 2^2 factorials for each pair of factors while holding remaining factors at the center level. Adds center-point replicates.
Estimates	All main effects, all 2-factor interactions, all quadratic terms. Full second-order model.

definitive_screening

Definitive Screening Design (DSD)

PROPERTY	VALUE
Operation string	`"definitive_screening"`
Purpose	Modern screening design that can detect curvature and some 2-factor interactions, unlike traditional screening designs.
Requirements	3+ factors with exactly 2 levels (continuous). Internally uses 3 levels: low, center, high.
Run count	2k + 1 runs for k factors (e.g., 13 runs for 6 factors).
Algorithm	Conference matrix construction. Each pair of columns has the property that when one factor is at its center, the other varies across all three levels.
Estimates	Main effects (unaliased with 2-factor interactions). Quadratic effects (detects curvature). Some 2-factor interactions (with k ≥ 6).

taguchi

Taguchi Orthogonal Array

PROPERTY	VALUE
Operation string	`"taguchi"`
Purpose	Robust design methodology. Optimizes signal-to-noise (S/N) ratios to find settings that are robust to noise factors.
Requirements	2+ factors, 2+ levels each. Automatically selects the appropriate orthogonal array (L4, L8, L9, L12, L16, L18, L27, etc.).
Run count	Determined by the selected orthogonal array. Depends on factor count and level count.
Algorithm	Selects the smallest standard orthogonal array that can accommodate all factors. Assigns factors to array columns.
Estimates	Main effects. S/N ratios (larger-is-better, smaller-is-better, nominal-is-best). Limited interaction estimation depending on the array.

d_optimal

D-Optimal Design

PROPERTY	VALUE
Operation string	`"d_optimal"`
Purpose	Algorithmically constructed design that maximizes the determinant of the information matrix. Useful for irregular design spaces and custom run budgets.
Requirements	2+ factors, 2+ levels each. Works with any combination of factor types.
Run count	User-specified or auto-calculated. Must be at least p (number of model parameters).
Algorithm	Fedorov coordinate-exchange algorithm. Starts from a candidate set of all possible combinations, iteratively swaps points to maximize $\|X'X\|$.
Estimates	Main effects and interactions (depending on the model specified). Optimized for the assumed model.

mixture_simplex_lattice

Mixture Simplex-Lattice Design

PROPERTY	VALUE
Operation string	`"mixture_simplex_lattice"`
Purpose	Formulation experiments where components must sum to a constant (e.g., 1 or 100%). Explores the simplex design space on a regular lattice grid.
Requirements	2+ factors (mixture components). Levels define proportion increments.
Run count	C(q + m - 1, m) where q = number of components, m = lattice degree.
Algorithm	Generates all lattice points on the simplex where each component takes values 0, 1/m, 2/m, ..., 1 and all components sum to 1.
Estimates	Linear and interaction blending effects in Scheffé polynomial models.

mixture_simplex_centroid

Mixture Simplex-Centroid Design

PROPERTY	VALUE
Operation string	`"mixture_simplex_centroid"`
Purpose	Formulation experiments. Uses centroid points of the simplex for a more focused exploration than the lattice design.
Requirements	2+ factors (mixture components).
Run count	2^q - 1 points (vertices, edge midpoints, face centroids, overall centroid).
Algorithm	Generates all subsets of components: pure components (vertices), binary blends (edge midpoints), ternary blends (face centroids), up to the overall centroid where all components are equal.
Estimates	Full Scheffé polynomial up to the q-th degree blending terms.

linear_sweep

Linear Parameter Sweep

PROPERTY	VALUE
Operation string	`"linear_sweep"`
Purpose	Characterize a single factor's effect across its range with evenly spaced points.
Requirements	Exactly 1 continuous factor with 2 levels (defining min and max).
Run count	Configurable via `sweep_points`. Default: auto.
Algorithm	Generates linearly spaced points from the low level to the high level.
Estimates	Response curve shape. Enables knee-point detection and saturation analysis.

log_sweep

Logarithmic Parameter Sweep

PROPERTY	VALUE
Operation string	`"log_sweep"`
Purpose	Characterize a single factor across a wide dynamic range with logarithmically spaced points.
Requirements	Exactly 1 continuous factor with 2 levels (both positive). Defines min and max on a log scale.
Run count	Configurable via `sweep_points`. Default: auto.
Algorithm	Generates points spaced evenly on a $\log_{10}$ scale between the low and high levels.
Estimates	Response curve shape on a logarithmic axis. Useful for parameters spanning orders of magnitude.

i

Choosing a Design

Start with **full_factorial** if you have ≤ 4 factors with 2 levels. Use **plackett_burman** or **definitive_screening** to screen many factors. Move to **central_composite** or **box_behnken** for optimization. Use **latin_hypercube** for space-filling exploration of continuous spaces.

Part 4: Analysis Output

The `doe analyze` command produces statistical tables, effect estimates, and diagnostic metrics. This section documents every output structure.

ANOVA Table

The Analysis of Variance table decomposes total variability into components attributable to each factor and error.

COLUMN	DESCRIPTION
Source	Factor name, interaction term (e.g., `A:B`), or `Error` / `Residual`.
DF	Degrees of freedom. For a factor with L levels: DF = L - 1. For interactions: product of individual DFs. Error DF = N - (total model DF) - 1.
SS	Sum of squares. The portion of total variability explained by this source.
MS	Mean square = SS / DF.
F	F-statistic = MS_{source} / MS_{error}. Measures the ratio of explained to unexplained variance.
p-value	Probability of observing the F-statistic under the null hypothesis. Values < 0.05 are typically considered significant.

Error Estimation Methods

When there are no replicates, error must be estimated by alternative methods:

METHOD	WHEN USED	DESCRIPTION
Replicate error	block_count > 1 or center points present	Pure error from replicated runs. The gold standard.
Pooled higher-order	Full/fractional factorials without replicates	Pools the smallest effects (assumed negligible) into an error term. Uses the sum of squares from the smallest 1/3 of interaction terms.
Lenth's PSE	Unreplicated 2-level designs	Pseudo Standard Error based on the median of absolute effect estimates. Robust to active effects.

Effect Estimates

For each factor and response, the analysis reports:

OUTPUT	DESCRIPTION
Main effect magnitude	For 2-level factors: mean(high) - mean(low). For multi-level: max(level means) - min(level means). See formulas.
Standard error (SE)	Estimated standard error of the effect. Computed from MS_{error} and the design matrix.
Confidence interval	Effect ± $t_{\alpha/2,\ df}$ × SE. Default α = 0.05 (95% CI).
Contribution %	100 × SS_{factor} / SS_{total}. Percentage of total variability attributable to this factor.
Significance	p-value from the F-test. Marked as significant if $p < 0.05$.

Interaction Effects

Two-factor interaction effects measure how the effect of one factor depends on the level of another. Reported for designs with sufficient degrees of freedom (full factorial, CCD, Box-Behnken, etc.).

- **Interaction magnitude**: half the difference between the effect of factor A at high B vs. low B.
- **Interaction plot**: non-parallel lines indicate a significant interaction.
- **Aliasing**: in fractional/PB designs, interactions may be aliased with main effects. The alias structure is reported.

Summary Statistics

For each factor/level combination, the analysis computes:

- Mean response at each level
- Standard deviation at each level (when replicates exist)
- Min and max response at each level
- Sample count per level

- Best level (highest mean for maximize, lowest for minimize)

Ordinal Trend Analysis

For factors with `"type": "ordinal"`, additional trend analysis is performed:

- **Monotonic trend test**: determines if the response increases or decreases consistently with the ordinal levels.
- **Trend direction**: increasing, decreasing, or non-monotonic.
- **Trend strength**: Spearman rank correlation between level order and response mean.

Knee Point Detection

When `--knee` is specified (or for sweep designs), the analysis identifies the point of diminishing returns:

- **Knee point location**: the factor value where the response curve transitions from steep to flat.
- **Algorithm**: maximum curvature method. Finds the point where the second derivative magnitude is largest.
- **Saturation percentage**: what fraction of the total response range is achieved at the knee point.

Model Diagnostics

When a regression model is fitted (RSM designs, LHS), the following diagnostics are reported:

METRIC	DESCRIPTION	IDEAL
R^2	Coefficient of determination. Proportion of variance explained by the model.	> 0.80
Adjusted R^2	R^2 penalized for number of terms. Decreases if insignificant terms are added.	> 0.75
PRESS	Predicted Residual Error Sum of Squares. Leave-one-out cross-validation metric.	Small relative to SS_{total}
Predicted R^2	1 - PRESS / SS_{total}. Measures predictive ability on held-out points.	> 0.70, within 0.2 of Adj R^2
Leverage (h_{ii})	Diagonal of the hat matrix $H = X(X'X)^{-1}X'$. Identifies influential runs.	< 2p/n (where p = parameters, n = runs)

Part 5: Output Formats

Result JSON

Each test script invocation must output a JSON file containing the measured response values. This is the contract between the test script and the analysis engine.

Result JSON (single run)

```
{
  "yield": 85.3,
  "purity": 99.1,
  "cost": 12.50
}
```

- Keys must match the `name` fields in the `responses` array.
- Values must be numeric (int or float).
- The file is written to `out_directory/run_NNN.json` where NNN is the zero-padded run number.
- If the test script fails, no result file should be written (the run is marked as failed).

Design Matrix JSON

The `design_matrix.json` file is generated alongside the runner script. It contains the complete design in a machine-readable format.

```
design_matrix.json structure

{
  "metadata": { "name": "...", "design_type": "full_factorial", "n_runs": 8 },
  "factors": ["temperature", "pressure", "catalyst"],
  "runs": [
    { "run": 1, "temperature": "80", "pressure": "2.0", "catalyst": "A" },
    { "run": 2, "temperature": "60", "pressure": "2.0", "catalyst": "B" },
    ...
  ]
}
```

CSV & TSV Export

The `doe export-data` command produces a flat file with one row per run.

```
CSV format

run,temperature,pressure,catalyst,yield,purity,cost
1,80,2.0,A,85.3,99.1,12.50
2,60,2.0,B,72.1,98.5,10.20
...
```

- First column is always `run` (1-based run number in execution order).
- Factor columns appear in definition order.
- Response columns appear after factor columns, in definition order.
- TSV format uses tab separators instead of commas.
- With `--partial`, missing response values are left empty.

Worksheet Formats

The `doe export-worksheet` command produces a blank template for manual data recording.

CSV Worksheet

```csv
run,temperature,pressure,catalyst,yield,purity,cost
1,80,2.0,A,,,
2,60,2.0,B,,,
...
```

Markdown Worksheet

```markdown
| run | temperature | pressure | catalyst | yield | purity | cost |
|-----|-------------|----------|----------|-------|--------|------|
| 1   | 80          | 2.0      | A        |       |        |      |
| 2   | 60          | 2.0      | B        |       |        |      |
...
```

HTML Report

The analysis HTML report (`doe report` or `doe analyze`) is a self-contained file that includes:

- **Experiment summary**: metadata, design type, factor/response definitions, run count.
- **Design matrix**: full table of all runs and their factor settings.
- **ANOVA tables**: one per response variable.
- **Effect estimates**: main effects and interactions with confidence intervals.
- **Plots** (embedded as base64 PNG): Pareto chart, main effects plots, interaction plots, normal/half-normal probability plots, residual diagnostics.
- **Optimization results**: optimal factor settings for each response.
- **Design evaluation**: D-efficiency, A-efficiency, G-efficiency metrics.

Part 6: Statistical Formulas

This section documents the key formulas used by the analysis engine. All computations follow standard DOE textbook conventions (Montgomery, Box-Hunter-Hunter).

Effect Calculation

Two-level factors

$$\text{Effect} = \text{mean}(Y_{high}) - \text{mean}(Y_{low})$$

The main effect is the difference in average response between the high (+1) and low (-1) levels, averaged over all other factors.

Multi-level factors

$$\text{Effect} = \max(\bar{Y}_1, \bar{Y}_2, \ldots, \bar{Y}_L) - \min(\bar{Y}_1, \bar{Y}_2, \ldots, \bar{Y}_L)$$

For factors with more than 2 levels, the effect is the range of the level means — the maximum average response minus the minimum average response.

Interaction effect (two-level)

$$\text{AB interaction} = \tfrac{1}{2} \left[(\text{mean}(Y_{A+,B+}) + \text{mean}(Y_{A-,B-})) - (\text{mean}(Y_{A+,B-}) + \text{mean}(Y_{A-,B+})) \right]$$

ANOVA Decomposition

$$SS_{total} = \sum_i (Y_i - \bar{Y})^2$$

$$SS_{factor} = \sum_{j=1}^{L} n_j (\bar{Y}_j - \bar{Y})^2$$

$$MS_{factor} = SS_{factor} / DF_{factor}$$

$$F = MS_{factor} / MS_{error}$$

Where $\bar{Y}_j$ is the mean response at level j, n_j is the count at level j, and $\bar{Y}$ is the grand mean. The p-value is computed from the F-distribution with (DF_{factor}, DF_{error}) degrees of freedom.

Lenth's Pseudo Standard Error

$$s_0 = 1.5 \times \text{median}(|c_1|, |c_2|, ..., |c_m|)$$

$$PSE = 1.5 \times \text{median}(|c_i| : |c_i| < 2.5 \times s_0)$$

Where c_i are the contrast (effect) estimates. Lenth's method is used for unreplicated 2-level factorial designs where there is no independent estimate of error. The first step computes a preliminary estimate s_0, then the PSE refines it by excluding effects larger than $2.5 \times s_0$ (likely active effects).

Confidence Intervals

$$CI = \text{effect} \pm t_{\alpha/2,\, df_{error}} \times SE$$

$$SE = \sqrt{(MS_{error} \times (1/n_+ + 1/n_-))}$$

For balanced 2-level designs: $SE = \sqrt{(4 \times MS_{error} / N)}$, where N is the total number of runs. The t-quantile uses the degrees of freedom from the error term.

D-efficiency

$$\text{D-efficiency} = 100 \times (|X'X| \,/\, n)^{1/p} \,/\, n$$

Where X is the model matrix (n × p), n is the number of runs, and p is the number of model parameters. A D-efficiency of 100% means the design is D-optimal for the assumed model. See Part 7 for interpretation.

Desirability Functions

Individual desirability (maximize)

$$d_i = ((y_i - \text{worst}) \,/\, (\text{best} - \text{worst}))^s$$

Where `worst` and `best` come from the `bounds` field on the response. The exponent s controls the shape: s = 1 (linear), s > 1 (emphasis on reaching best), s < 1 (diminishing returns near best). Default s = 1. Values below worst get d = 0; values above best get d = 1.

Individual desirability (minimize)

$$d_i = ((\text{worst} - y_i) \,/\, (\text{worst} - \text{best}))^s$$

For minimize, `worst` > `best`. Values above worst get d = 0; values below best get d = 1.

Overall desirability

$$D = (\prod d_i^{w_i})^{1 / \sum w_i}$$

The weighted geometric mean of individual desirabilities. If any single $d_i = 0$, the overall D = 0 regardless of other responses. Weights w_i come from the `weight` field on each response.

Part 7: Design Evaluation Metrics

These metrics quantify how well a design supports parameter estimation. They are reported by `doe info` and included in analysis reports.

D-efficiency

$$\text{D-efficiency} = 100 \times (|X'X|^{1/p}) / n$$

Interpretation: D-efficiency measures the overall precision of parameter estimates. It is proportional to the p-th root of the determinant of the information matrix X'X, normalized by the number of runs. Higher values mean smaller confidence ellipsoids for the estimated coefficients.

RANGE	INTERPRETATION
90–100%	Excellent. Near-optimal for parameter estimation.
70–89%	Good. Suitable for most applications.
50–69%	Moderate. Consider augmenting the design.
< 50%	Poor. The design may not reliably estimate all parameters.

> ℹ Orthogonal designs (full factorial, Plackett-Burman) achieve 100% D-efficiency for the main-effects model. D-optimal designs maximize this metric by construction.

A-efficiency

$$\text{A-efficiency} = 100 \times p / \text{trace}((X'X)^{-1})$$

Interpretation: A-efficiency measures the average variance of parameter estimates. It is inversely proportional to the trace (sum of diagonal elements) of $(X'X)^{-1}$. Minimizing the trace minimizes the average variance across all coefficients.

- A-efficiency focuses on *average* precision, while D-efficiency focuses on *overall* precision.
- A design can have high D-efficiency but lower A-efficiency if some parameters are estimated much more precisely than others.
- For orthogonal designs, A-efficiency = D-efficiency = 100%.

G-efficiency

$$\text{G-efficiency} = 100 \times p / (n \times \max(h_{ii}))$$

Where h_{ii} are the diagonal elements of the hat matrix $H = X(X'X)^{-1}X'$.

Interpretation: G-efficiency measures the worst-case prediction variance across the design space. It is related to the maximum leverage point. A design with high G-efficiency has relatively uniform prediction variance — no single point dominates the model.

- G-efficiency = 100% means all design points have equal leverage (balanced design).
- Lower G-efficiency indicates that some regions of the factor space are predicted with much less precision than others.
- Particularly important for response surface designs where prediction across the entire space matters.

⚠

No Single Metric Is Sufficient

Always examine D-, A-, and G-efficiency together. A design optimized for one criterion may perform poorly on another. For most practical purposes, D-efficiency above 70% with G-efficiency above 50% indicates an adequate design.

← User Guide

Theory →

PART IV

Design of Experiments

A Practical Guide — From Theory to Automated Analysis

Preface

> *This book teaches the theory and practice of Design of Experiments (DOE) — a systematic method for planning experiments that extracts the maximum information from the minimum number of runs.*

It is written for engineers, scientists, and anyone who runs experiments, benchmarks, or tests and wants to do so more efficiently.

DOE is not new. Ronald Fisher developed the foundations in the 1920s and 1930s for agricultural experiments at Rothamsted. George Box, Genichi Taguchi, and others extended it to manufacturing, quality engineering, and process optimization. Yet the technique remains underused, especially in software engineering and technology, where practitioners often default to changing one variable at a time or running exhaustive grid searches.

This guide takes a practical approach. Each chapter introduces the theory you need, then shows you how to apply it. Mathematical derivations are included where they build intuition, but the emphasis is on understanding *when* and *why* to use each technique, not on proofs.

Who This Book Is For

Engineers optimizing systems, processes, or software configurations. Scientists planning laboratory or field experiments. Data scientists evaluating model hyperparameters. Quality engineers improving manufacturing processes. Anyone who has ever wondered: *"Which of these 10 knobs actually matters?"*

Prerequisites: Basic familiarity with mean, variance, and standard deviation. No prior statistics coursework is assumed.

How This Book Is Organized

Part I (Chapters 1–2) lays the conceptual and statistical groundwork. You'll learn why planned experiments outperform ad hoc testing and acquire the vocabulary needed for the rest of the book.

Part II (Chapters 3–6) covers the major design families — full factorials, fractional factorials, screening designs, and response surface designs. Each chapter explains when to use the design, how the mathematics works, and walks through a realistic example.

Part III (Chapters 7–9) focuses on analysis: interpreting Pareto charts, fitting response surface models, diagnosing problems, and making data-driven decisions.

The **User Guide** at the end is a complete software reference for the DOE Helper Tool, including configuration fields, CLI commands, and step-by-step tutorials.

> 📖 **Reading Order**
>
> If you're new to DOE, read Chapters 1–3 first, then jump to Chapter 7 to learn analysis. Come back to Chapters 4–6 when you need designs beyond the basic factorial. If you already know DOE and just want to use the tool, skip straight to the User Guide.

Chapter 1

Why Design of Experiments?

The Cost of Unplanned Experiments

Every experiment costs something — time, money, materials, compute cycles. A poorly planned experiment wastes these resources and may lead to wrong conclusions. A well-designed experiment extracts the maximum learning from each run.

> 🍰 **The Cake Analogy**
>
> Imagine you're baking a cake and want to make it taste better. You could try changing the sugar amount, then the oven temperature, then the baking time — one thing at a time. After dozens of attempts, you might find a decent recipe. But you would never discover that a *specific combination* of less sugar *with* higher temperature produces the best result, because you never tested them together. DOE solves exactly this problem.

Consider a team tuning a database. They have five configuration knobs they suspect affect performance: buffer pool size, I/O capacity, connection limit, query cache, and thread pool size. The one-variable-at-a-time (OVAT) approach would be:

1. Fix all knobs at their defaults.
2. Change buffer pool size. Measure.
3. Reset buffer pool. Change I/O capacity. Measure.
4. Continue for each knob.

This requires at least 10 runs (2 per knob) and only measures each knob's effect *in isolation, at the default settings of every other knob*. It tells you nothing about how the knobs interact.

A Plackett-Burman screening design tests all five knobs in just **8 runs** and estimates every main effect simultaneously. A full factorial on the two most important knobs then takes 4 more runs and reveals their interaction. Twelve runs total, and you know more than OVAT would teach you in fifty.

The Three Principles of DOE

Fisher established three principles that remain the foundation of every well-designed experiment:

1. Randomization

Randomly assign the order in which experimental runs are performed to protect against unknown lurking variables. If you run all low-temperature experiments in the morning and all high-temperature experiments in the afternoon, any morning-to-afternoon drift becomes confounded with your temperature factor.

Randomization does not eliminate lurking variables — it distributes their effects equally across all treatments, on average. This ensures that the statistical tests remain valid.

2. Replication

Repeat experimental runs to estimate experimental error and increase the precision of effect estimates. A single run at each combination tells you the response, but not how much it varies.

Replication vs. Repetition

Replication means independently setting up and running the experiment again. **Repetition** means measuring the same setup multiple times. Repetition estimates measurement error, but replication estimates the full experimental error. DOE requires replication.

3. Blocking

Group experimental runs into blocks of similar conditions to remove known sources of variation from the experimental error. If you must run experiments over two days, make each day a block.

The OVAT Trap: A Quantitative Example

Suppose the true relationship between yield and two factors (temperature T and pressure P) is:

$$Y = 50 + 10T + 5P + 8TP + \varepsilon$$

where $T, P \in \{-1, +1\}$ (coded levels) and $\varepsilon \sim N(0, 1)$.

× OVAT Approach

Run at T=-1, P=-1: Y = 43

Change T to +1: Y = 47

Conclusion: Temperature effect = 4

Wrong! True effect is 10.

✓ Factorial Approach

All 4 combinations tested.

$ME_T = (47+73)/2 - (43+37)/2 = 20$

Effect = 20/2 = **10** ✔

Correct! Also finds interaction = 8.

The OVAT estimate is biased because it was measured at P=-1, where the interaction TP reduces temperature's apparent effect. The factorial design correctly estimates both effects *and* their interaction, using the same total data.

Hidden Replication in Factorials

In a 2^k factorial, every effect estimate uses *all* 2^k observations. Each observation plays a role in estimating every effect. This is why factorials are so efficient — there is no wasted data.

A Concrete Example: Web Server Tuning

Suppose you are optimizing a web server. Two factors matter: **thread count** (10 or 100) and **cache size** (64 MB or 512 MB). The true performance:

THREADS	CACHE	THROUGHPUT (REQ/SEC)
10	64 MB	500
100	64 MB	600
10	512 MB	700
100	512 MB	**1,400**

OVAT Result

Starting at (10, 64MB):

Threads: 500 → 600 (+100)

Cache: 500 → 700 (+200)

Recommendation: Use 512MB cache alone.

Predicted: 700 req/sec

DOE Result

All 4 combinations tested:

Thread effect: +400

Cache effect: +500

Interaction: +600!

Recommendation: Both high.

Actual: 1,400 req/sec

OVAT would have left **700 req/sec on the table** — a 2x performance improvement, missed entirely because OVAT can't detect interactions.

Effect Sparsity: Why DOE Works

A fundamental observation underpins DOE's practicality: in most real systems, only a small fraction of factors truly matter. This is called the **effect sparsity principle** (sometimes the Pareto principle of effects). Out of 20 possible factors, typically 3–5 drive 80% or more of the variation.

This means you can afford a two-stage strategy: screen many factors cheaply to find the vital few, then investigate those few in detail. The total cost is far less than testing every factor exhaustively.

Effect Hierarchy

A related principle: **main effects** tend to be larger than **two-factor interactions**, which tend to be larger than **three-factor interactions**, and so on. This hierarchy is why fractional factorial designs work — they sacrifice higher-order interaction estimates that are usually negligible anyway.

When NOT to Use DOE

DOE is not always the right tool. Consider alternatives when:

- **You have only one factor.** A simple A/B test or one-way comparison is sufficient.
- **The response is not measurable.** DOE requires a quantifiable outcome. Subjective preferences need special treatment (conjoint analysis, paired comparisons).
- **The system is completely unknown.** If you have no idea which factors might matter, start with domain-expert brainstorming and a few exploratory runs before committing to a formal design.
- **Runs are effectively free.** If you can test millions of combinations cheaply (e.g., a fast simulation), a random search or grid search may be simpler.

In most practical engineering situations, however, experiments have a non-trivial cost, and DOE will save you time and resources.

Chapter 2

Statistical Foundations

The Linear Model

DOE assumes the response can be modeled as a linear combination of factor effects:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_{12} X_1 X_2 + \varepsilon$$

where β_0 is the grand mean, β_i are main effect coefficients, β_{ij} are interaction coefficients, and ε is random error.

This model is "linear" in the *parameters* (β), not necessarily in the factors. Even quadratic terms like X^2 are linear in the coefficient β_{ii}.

The Full Model for Three Factors

For 3 factors (A, B, C) with interactions and quadratic terms, the complete model is:

$$Y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_{12} x_1 x_2 + \beta_{13} x_1 x_3 + \beta_{23} x_2 x_3 + \beta_{123} x_1 x_2 x_3 + \varepsilon$$

That's 8 terms (including the intercept) for 3 factors — exactly the number of runs in a 2^3 factorial. This isn't a coincidence: each run provides one equation, and the design is constructed so that the 8 equations are independent (orthogonal). When we add quadratic terms $\beta_{11} x_1^2$, $\beta_{22} x_2^2$, $\beta_{33} x_3^2$, the model grows to 11 terms, requiring more than 8 runs — this is why response surface designs (CCD, Box-Behnken) need more runs than factorials.

Linear model (factorial)

1 + k + C(k,2) + ... terms

For k=3: **8 terms, 8 runs**

Captures main effects + interactions

Quadratic model (RSM)

Linear + k quadratic terms

For k=3: **10–11 terms, 15–20 runs**

Also captures curvature

Coded Variables

We convert natural factor values to coded values (-1, +1) using:

$$x = (X - \bar{X}) / (X_{range} / 2)$$

This standardizes all factors to the same scale, making effect magnitudes directly comparable. A factor with levels 150°C and 200°C has coded values -1 and +1, where 175°C is zero.

📏 **Why Code?**

Imagine comparing the "effect" of temperature (150 vs 200) to pressure (2 vs 6). In natural units, the numbers aren't comparable. In coded units, both range from -1 to +1, so an effect of 10 in temperature is directly comparable to an effect of 5 in pressure.

Worked Example: Coding Factor Values

Consider a temperature factor with levels 150°C (low) and 200°C (high):

NATURAL VALUE	CALCULATION	CODED VALUE
150°C	(150 - 175) / 25	**-1**
175°C	(175 - 175) / 25	**0**
200°C	(200 - 175) / 25	**+1**
162.5°C	(162.5 - 175) / 25	-0.5
212.5°C	(212.5 - 175) / 25	+1.5 (star point)

The center ($\bar{X}$) is (150 + 200) / 2 = 175, and the half-range is (200 - 150) / 2 = 25. Star points in CCD designs produce coded values beyond ±1, which is how they sample outside the original range.

Why Coding Matters for the DOE Helper Tool

The RSM fitting in `doe optimize` automatically codes all continuous factors to [-1, +1] before fitting. This means the reported coefficients are directly comparable in magnitude — a coefficient of 10 for temperature is twice as important as a coefficient of 5 for pressure, regardless of their natural units. Categorical factors are coded as -1 (first level) and +1 (last level).

Estimating Effects

Main Effects

The main effect of factor A is the difference between the average response at the high level and the average response at the low level:

$$ME_A = \bar{Y}_{A+} - \bar{Y}_{A-}$$

Interaction Effects

The interaction effect of factors A and B measures whether the effect of A depends on the level of B:

$$IE_{AB} = ½[(\bar{Y}_{A+,B+} - \bar{Y}_{A-,B+}) - (\bar{Y}_{A+,B-} - \bar{Y}_{A-,B-})]$$

If this is zero, A and B act independently. If large, their combined effect is synergistic (positive) or antagonistic (negative).

Percentage Contribution

To compare the relative importance of effects, we compute:

$$\%\ Contribution_i = |Effect_i| / \Sigma|Effect_j| \times 100$$

This is what the Pareto chart displays — a ranked bar chart of contributions with a cumulative line.

📊 **Reading a Pareto Chart**

The DOE Helper Tool generates Pareto charts automatically. The left axis shows effect magnitude as bars (tallest first). The right axis shows a cumulative percentage line. Factors to the left of where the cumulative line reaches ~80% are the "vital few" — the factors that drive most of the variation. Factors to the right are the "trivial many" that can often be ignored or fixed at convenient levels.

Confidence Intervals

A 95% confidence interval on a main effect is:

$$\text{Effect} \pm t_{\alpha/2,\, df} \times SE_{effect}$$

If the interval includes zero, the effect is not statistically significant at the 5% level. The DOE Helper Tool computes these automatically using the t-distribution.

Practical vs. Statistical Significance

A statistically significant effect might be too small to matter in practice. Conversely, a practically important effect might not reach statistical significance with few observations. Always consider both.

What Affects the Width of Confidence Intervals?

The confidence interval width depends on three things:

FACTOR	EFFECT ON CI WIDTH	HOW TO IMPROVE
Number of runs (N)	More runs → narrower CIs	Add replicates via `block_count`
Experimental noise (σ)	Higher noise → wider CIs	Improve measurement precision, control nuisance variables
Confidence level	99% → wider than 95%	Accept 95% (standard practice)

🔍 **The Microscope Analogy**

Think of confidence intervals as the resolution of a microscope. More replicates give you a higher-power lens — you can see smaller effects. With too few runs, only the biggest effects stand out above the noise floor. This is why the screening-to-optimization pipeline works: use PB to find the big effects, then use RSM with more runs per factor to precisely measure them.

Orthogonality and Balance

A design is **orthogonal** when the columns of the design matrix are uncorrelated. This means each effect estimate is independent of every other — changing one factor's range or removing one effect from the model doesn't change the other estimates.

A design is **balanced** when each factor level appears equally often. Balance ensures that effect estimates use all available data symmetrically.

⚖ **The Scale Analogy**

Think of balance like a weighing scale. If you test factor A at the high level 6 times but at the low level only 2 times, your estimate of A's effect is skewed — it's like putting heavier weights on one side of the scale. Balanced designs put equal weight on both sides.

Full factorial and Plackett-Burman designs are both orthogonal and balanced. This is what makes them statistically powerful — every run contributes useful information to every effect estimate.

Visualizing Orthogonality

In an orthogonal design, knowing the level of one factor tells you nothing about the level of any other factor. Here's the contrast:

Not orthogonal

When A is high, B is always high. The effects of A and B are confounded — you can't tell which one caused the change.

RUN	A	B
1	-1	-1
2	+1	+1
3	-1	-1
4	+1	+1

Orthogonal

A and B vary independently. Each effect is estimated without contamination from the other.

RUN	A	B
1	-1	-1
2	+1	-1
3	-1	+1
4	+1	+1

Degrees of Freedom

Degrees of freedom (df) determine how much information your design provides. In a 2^k factorial with $N = 2^k$ runs:

- **1 df** for the overall mean
- **k df** for main effects
- **C(k,2) df** for two-factor interactions
- **Remaining df** available for error estimation (if you have replicates)

Without replicates, you must assume higher-order interactions are zero and "pool" their degrees of freedom to estimate error. This is why center-point replicates are valuable — they provide pure-error degrees of freedom without increasing the factorial portion of the design.

Worked Example: Degree of Freedom Budget

For a 2^4 factorial (16 runs) with 2 replicates (32 total runs):

SOURCE	DF	WHAT IT ESTIMATES
Mean	1	Overall average
Main effects (A, B, C, D)	4	Individual factor contributions
2-factor interactions (AB, AC, ...)	6	Pairwise synergies/antagonisms
3-factor interactions (ABC, ...)	4	Usually negligible (pooled to error)
4-factor interaction (ABCD)	1	Usually negligible (pooled to error)
Pure error (from replication)	16	Experimental noise
Total	**32**	

The 16 error degrees of freedom come from the replicate: each of the 16 factorial points has 2 observations, giving 16 × (2-1) = 16 df for pure error. Without replication (single replicate), you'd need to assume higher-order interactions are negligible and pool their df to estimate error.

The Unreplicated Factorial Dilemma

With no replicates, there's no independent error estimate. You must either (a) assume high-order interactions are zero and use their degrees of freedom for error, (b) use the normal probability plot to visually separate real effects from noise, or (c) add center-point replicates. The DOE Helper Tool supports option (a) automatically and option (c) via blocking.

The Normal Probability Plot

When you have no replicates (and therefore no independent error estimate), a **normal probability plot** of effects helps identify which effects are real. Negligible effects cluster along a straight line (they are just noise). Effects that deviate from the line are likely real.

Reading a Normal Probability Plot

Points on the line: Noise — these effects are indistinguishable from random error.
Points far from the line: Real effects — the further from the line, the more confident you can be.
This technique was developed by Cuthbert Daniel (1959) and remains one of the most useful diagnostic tools in DOE.

Statistical Power and Sample Size

Power is the probability that your experiment will detect a real effect of a given size. Low power means you might run an experiment and conclude "nothing matters" when, in fact, something does — you just didn't have enough data to see it.

POWER	MEANING	RUNS NEEDED (TYPICAL)
50%	Coin flip — might miss real effects	Unreplicated factorial
80%	Standard target — acceptable for most work	2 replicates
90%	High confidence — for critical decisions	3–4 replicates

The key insight: **power depends on the signal-to-noise ratio**. If your factor ranges produce large effects relative to experimental noise, even a small design has high power. If the effects are subtle, you need more replicates. This is why factor range selection (Chapter 1) is so important — wider ranges amplify the signal.

Practical Rule of Thumb

You can detect an effect that is at least 2–3 times the standard deviation of your experimental noise. If a previous study or pilot run suggests $\sigma = 5$, then you can reliably detect effects of 10–15 or larger. Effects smaller than that will need more replicates or reduced noise to be detected.

Chapter 3

Full Factorial Designs

> *The 2^k factorial tests every combination of k factors at 2 levels each. It is the most complete design — and the foundation for all others.*

The 2^k Factorial

For k factors, each at 2 levels, we need 2^k runs:

4	8	16	32	64	1024
2 factors	3 factors	4 factors	5 factors	6 factors	10 factors

Growth is exponential. This is manageable for 2–5 factors, but becomes impractical beyond that — which motivates fractional and screening designs.

Understanding Exponential Growth

The run count grows faster than most people intuit. Here's a comparison of run count vs. information gained:

FACTORS (K)	RUNS (2^K)	MAIN EFFECTS	2-FACTOR INTERACTIONS	TOTAL EFFECTS ESTIMATED
2	4	2	1	3
3	8	3	3	7
4	16	4	6	15
5	32	5	10	26
6	64	6	15	57
7	128	7	21	120

At k=5, only 15 of the 31 estimable effects are main effects or two-factor interactions — the rest are higher-order interactions that are usually negligible. This is the fundamental insight that motivates fractional designs: you're paying for information you probably don't need.

> 💰 **The Budget Analogy**
>
> Think of each experimental run as spending money. A full factorial spends evenly across all possible effects, including three-way, four-way, and higher interactions. That's like buying comprehensive insurance on a car, house, boat, and spaceship when you only own a car. Fractional designs spend the same budget more wisely by focusing on the effects most likely to matter.

The Design Matrix

A 2^3 full factorial design matrix:

RUN	A	B	C
1	-1	-1	-1
2	+1	-1	-1
3	-1	+1	-1
4	+1	+1	-1
5	-1	-1	+1
6	+1	-1	+1
7	-1	+1	+1
8	+1	+1	+1

Every column is balanced: equal numbers of -1 and +1. Every pair of columns is orthogonal. This ensures unbiased, independent estimates of all effects.

How to Read a Design Matrix

Each row is one experiment. Each column is one factor. The values -1 and +1 represent the low and high levels of that factor. To compute any interaction column, multiply the corresponding factor columns element-by-element:

RUN	A	B	AB	C	AC	BC	ABC
1	-1	-1	+1	-1	+1	+1	-1
2	+1	-1	-1	-1	-1	+1	+1
3	-1	+1	-1	-1	+1	-1	+1
4	+1	+1	+1	-1	-1	-1	-1
5	-1	-1	+1	+1	-1	-1	+1
6	+1	-1	-1	+1	+1	-1	-1
7	-1	+1	-1	+1	-1	+1	-1
8	+1	+1	+1	+1	+1	+1	+1

Notice that *every* column (including interaction columns) is balanced (four +1s and four -1s) and orthogonal to every other column. This is the mathematical foundation that makes effect estimation unbiased. The DOE Helper Tool constructs and displays this matrix with `doe info` and `doe generate --dry-run`.

When to Use Full Factorials

Use Full Factorial When:

- You have 2–4 factors (manageable run count)
- You need to estimate *all* interactions, not just main effects
- Each run is cheap (fast simulations, automated tests)
- You want zero ambiguity about which effects are real

When to Avoid Full Factorials

- More than 5 factors (run count explodes: 64+ runs)
- Runs are expensive or time-consuming
- You're still in the screening phase and don't know which factors matter
- You need more than 2 levels per factor (consider general factorial or RSM instead)

Worked Example: Packaging Seal Strength

Three factors at 2 levels each: Seal Temperature (A), Pressure (B), Dwell Time (C). Response: seal strength in Newtons.

RUN	A	B	C	STRENGTH (N)
1	-1	-1	-1	15.3
2	+1	-1	-1	27.1
3	-1	+1	-1	18.6
4	+1	+1	-1	32.4
5	-1	-1	+1	17.8
6	+1	-1	+1	29.5
7	-1	+1	+1	20.1
8	+1	+1	+1	34.2

Main effect of A (Temperature):

$$ME_A = (27.1 + 32.4 + 29.5 + 34.2)/4 - (15.3 + 18.6 + 17.8 + 20.1)/4 = 30.8 - 17.95 = \mathbf{12.85\ N}$$

Temperature is the dominant factor, increasing seal strength by ~12.85 N on average. The tool computes all effects, confidence intervals, and creates the Pareto chart automatically.

Complete Effect Analysis

Let's compute all effects from the seal strength data:

EFFECT	CALCULATION	VALUE	% CONTRIBUTION
A (Temperature)	(27.1+32.4+29.5+34.2)/4 - (15.3+18.6+17.8+20.1)/4	**12.85 N**	69.3%
B (Pressure)	(18.6+32.4+20.1+34.2)/4 - (15.3+27.1+17.8+29.5)/4	**3.65 N**	19.7%
C (Dwell Time)	(17.8+29.5+20.1+34.2)/4 - (15.3+27.1+18.6+32.4)/4	**2.05 N**	11.1%
AB	½[(32.4-18.6+34.2-20.1) - (27.1-15.3+29.5-17.8)]	0.65 N	—
AC	½[(29.5-27.1+34.2-32.4) - (17.8-15.3+20.1-18.6)]	0.15 N	—
BC	½[(20.1-18.6+34.2-32.4) - (17.8-15.3+29.5-27.1)]	0.15 N	—

Interpretation: Temperature (A) dominates at 69.3% contribution. Pressure (B) contributes 19.7%, and Dwell Time (C) accounts for 11.1%. All interaction effects are near zero — the factors act independently. The Pareto chart would show a steep drop after A, confirming that temperature is the primary lever for improving seal strength.

The 80/20 Rule in Action

In this example, one factor (A) accounts for ~70% of the total effect. This is typical in practice — most processes are dominated by one or two key factors. The power of factorial designs is that they reveal this unambiguously, including any interactions, in a single set of experiments.

Adding Center Points

A 2^k factorial only tests the extremes. What if the response curves between the extremes? Adding **center points** — runs at the midpoint of all factor ranges — lets you detect curvature without fitting a full quadratic model.

If the center-point average differs significantly from the average of the factorial points, curvature is present, and you should consider upgrading to a CCD or Box-Behnken design for the next phase.

Center Points Serve Two Purposes

1. Curvature detection: Compare the center-point mean to the factorial mean. A statistically significant difference signals nonlinearity.
2. Pure error estimation: Replicated center points provide degrees of freedom for error, even in unreplicated factorials. Three to five center replicates are typically sufficient.

Detecting Curvature with Center Points

Continuing the seal strength example: suppose we add 3 center-point replicates at Temperature=175°C, Pressure=4 bar, Dwell Time=7.5 sec. The results are: 28.1, 27.4, 28.8 N.

Center mean = (28.1 + 27.4 + 28.8) / 3 = **28.10 N**
Factorial mean = (15.3 + 27.1 + 18.6 + 32.4 + 17.8 + 29.5 + 20.1 + 34.2) / 8 = **24.38 N**

The center mean (28.10) is higher than the factorial mean (24.38), suggesting **positive curvature** — the response surface bows upward in the middle. The difference (3.72 N) is large enough to warrant investigating a quadratic model using CCD or Box-Behnken.

Center Points Only Detect Curvature — They Don't Model It

Center points tell you *whether* curvature exists, but not *which factor* is curved. To fit individual quadratic terms (x_1^2, x_2^2, etc.), you need a response surface design with at least 3 distinct levels per factor.

Blocking a Factorial Design

Sometimes you can't complete all runs under identical conditions. Perhaps the raw material comes in batches, or experiments span multiple days. **Blocking** partitions the runs into groups (blocks) so that known sources of variation are accounted for.

In a 2^k factorial, you assign the highest-order interaction to the block effect. For a 2^3 design split into 2 blocks, assign ABC to blocks: runs where ABC = -1 go in Block 1, and runs where ABC = +1 go in Block 2. The ABC interaction becomes confounded with the block effect, but all main effects and two-factor interactions remain estimable.

Choose Your Block Generator Wisely

Always confound the highest-order interaction with blocks. This sacrifices the least valuable information. Never confound a main effect or a two-factor interaction you care about with the block variable.

Worked Example: Blocking a 2^3 Design

We split the 2^3 seal strength design into 2 blocks. The generator is ABC (the three-factor interaction). Runs where A×B×C = -1 go in Block 1 (day 1); runs where A×B×C = +1 go in Block 2 (day 2):

Block 1 (A×B×C = -1)

RUN	A	B	C
1	-1	-1	-1
4	+1	+1	-1
6	+1	-1	+1
7	-1	+1	+1

Block 2 (A×B×C = +1)

RUN	A	B	C
2	+1	-1	-1
3	-1	+1	-1
5	-1	-1	+1
8	+1	+1	+1

Each block is balanced: within each block, every factor has an equal number of -1 and +1 values. This means all main effects (A, B, C) and two-factor interactions (AB, AC, BC) are estimable. Only the ABC interaction is confounded with the block effect — which is usually an acceptable trade-off.

Replicated Factorials

Running the entire factorial more than once gives you a direct estimate of experimental error and increases the precision of all effect estimates. With n replicates of a 2^k design, the standard error of each effect is:

$$SE_{effect} = 2s / \sqrt{(n \times 2^k)}$$

where s is the pooled standard deviation. Doubling the replicates reduces the standard error by a factor of √2 — you get about 41% more precision for twice the runs. This is diminishing returns, so most practitioners use 2–3 replicates rather than many.

Practical Guidance: Running a Full Factorial

Here's how a full factorial experiment typically proceeds with the DOE Helper Tool:

Full factorial workflow

```
# 1. Configure the experiment
$ cat config.json
{
  "factors": [
    {"name": "seal_temp", "levels": ["150", "200"], "type": "continuous",
"unit": "°C"},
    {"name": "pressure", "levels": ["2", "6"], "type": "continuous", "unit":
"bar"},
    {"name": "dwell_time", "levels": ["5", "10"], "type": "continuous",
"unit": "sec"}
  ],
  "responses": [{"name": "seal_strength", "optimize": "maximize", "unit":
"N"}],
  "settings": {"operation": "full_factorial", "block_count": 2}
}

# 2. Preview: 8 base runs × 2 blocks = 16 total runs
$ doe info --config config.json

# 3. Generate and run
$ doe generate --config config.json --seed 42
$ bash run_experiments.sh

# 4. Analyze
$ doe analyze --config config.json
$ doe optimize --config config.json
```

When to Stop

If the analysis reveals that one or two factors dominate (together accounting for 80%+ of the variation), and the interaction effects are small, you've found the vital few. Fix the unimportant factors at convenient levels and either declare victory or move to a response surface design to find the precise optimum of the important factors.

Chapter 4

Fractional Factorial Designs

> *When full factorials require too many runs, fractional factorials let you trade off complete interaction information for dramatic run-count reduction.*

The Problem of Exponential Growth

Full factorial runs grow as 2^k. For 7 factors, that's 128 runs. For 10, it's 1,024. This is often impractical.

But here's the saving insight: in most real systems, high-order interactions (three-factor and above) are negligible. The **effect sparsity principle** tells us that only a few effects — typically main effects and some two-factor interactions — drive the majority of the variation. A fractional factorial sacrifices information about high-order interactions (which are probably zero anyway) to dramatically reduce the run count.

✂ The Compression Analogy

A full factorial is like a raw, uncompressed image — it captures every pixel of information, including redundant detail you'll never notice. A fractional factorial is like JPEG compression: it discards the least important information (high-order interactions) while preserving the features that matter (main effects and key interactions). The "resolution" of the design tells you how aggressively you've compressed.

The 2^{k-p} Design

A fractional factorial runs 2^{k-p} experiments, where p is the number of "generators." A half-fraction (p=1) cuts runs by 50%. A quarter-fraction (p=2) cuts by 75%.

$2^{5-1} = 16$	$2^{7-2} = 32$	$2^{10-3} = 128$
vs. 32 full	vs. 128 full	vs. 1024 full

Resolution

Resolution describes the severity of the aliasing (confounding):

RESOLUTION	ALIASING	INTERPRETATION
III	Main effects aliased with 2-factor interactions	Good for screening; not for estimating interactions
IV	Main effects clear; 2-factor interactions aliased with each other	Good for most practical work
V+	Main effects and 2FIs clear	Nearly as good as full factorial

The Trade-Off

Fractional factorials save runs but lose information. In a Resolution III design, if factor A appears important, it might actually be the AB interaction that matters. Follow-up experiments (foldover) can resolve these ambiguities.

Resolution in Practice

Here's what each resolution level means concretely:

III

Main effects aliased with 2FIs. Suitable for screening only. Cannot separate A from BC.

IV

Main effects clear. 2FIs aliased with each other. Most practical choice — you know *which factors* matter but may not know *which pairs interact*.

V+

Main effects and all 2FIs clear. Nearly as informative as a full factorial. Rarely needed for screening.

The Minimum Acceptable Resolution

For screening (finding which factors matter): Resolution III is sufficient.
For characterization (understanding interactions): Resolution IV is the minimum.
For definitive studies (estimating all 2FIs): Resolution V or higher.

Generators and Defining Relations

A fractional factorial is constructed by writing k-p factors as a full factorial, then **generating** the remaining p factors as products of existing columns. For example, in a 2^{5-1} design, we might set E = ABCD. This is the **generator**.

The **defining relation** is I = ABCDE (multiply both sides by E). It tells you every alias pair. Any effect multiplied by the defining relation gives its alias:

$$A \times ABCDE = BCDE, \quad \text{so A is aliased with BCDE}$$
$$AB \times ABCDE = CDE, \quad \text{so AB is aliased with CDE}$$

In a Resolution V design like this, main effects are aliased only with four-factor interactions (negligible), and two-factor interactions are aliased with three-factor interactions (usually small). This is why Resolution V designs are nearly as good as full factorials.

Multiple Generators and Word Length

When p > 1 (quarter-fraction or smaller), you have multiple generators, and the defining relation contains multiple "words." For a 2^{7-2} design with generators F = ABCD and G = ABDE:

$$\text{Defining relation: } I = ABCDF = ABDEG = CEFG$$

The third word (CEFG) is the "generalized interaction" of the first two generators. The **minimum word length** in the defining relation determines the resolution. Here, the shortest word has 4 letters, so this is a Resolution IV design.

The DOE Helper Tool automatically computes and displays the alias structure when you use the `info` command with a fractional factorial design. This tells you exactly which effects are confounded with each other, so you can interpret your results correctly.

Minimum Aberration

Among all designs with the same resolution, the **minimum aberration** design has the fewest short words in its defining relation. This minimizes the number of low-order effects that are aliased together. The DOE Helper Tool uses minimum-aberration generators by default — the pyDOE3 library selects them automatically.

Worked Example: 2^{4-1} Half-Fraction

Four factors (A, B, C, D). A full factorial needs 16 runs. A half-fraction (p=1) needs only 8. We set D = ABC as the generator, giving defining relation I = ABCD (Resolution IV).

RUN	A	B	C	D=ABC
1	-1	-1	-1	-1
2	+1	-1	-1	+1
3	-1	+1	-1	+1
4	+1	+1	-1	-1
5	-1	-1	+1	+1
6	+1	-1	+1	-1
7	-1	+1	+1	-1
8	+1	+1	+1	+1

The alias structure: A+BCD, B+ACD, C+ABD, D+ABC, AB+CD, AC+BD, AD+BC. Main effects are clear of two-factor interactions (Resolution IV), but two-factor interactions are aliased in pairs (AB with CD, etc.).

Interpreting Aliased Effects

The alias pairs in our 2^{4-1} Resolution IV design are:

ESTIMATED EFFECT	ACTUALLY MEASURES	PRACTICAL IMPACT
A	A + BCD	Clean (BCD is negligible)
B	B + ACD	Clean (ACD is negligible)
C	C + ABD	Clean (ABD is negligible)
D	D + ABC	Clean (ABC is negligible)
AB	AB + CD	Ambiguous — could be either
AC	AC + BD	Ambiguous — could be either
AD	AD + BC	Ambiguous — could be either

Main effects are clean because they're aliased only with three-factor interactions (which are almost always negligible). But if the "AB" effect looks large, you can't tell whether it's really AB, or really CD, or some combination of both. This is where subject-matter expertise comes in — does an interaction between A and B make physical sense?

🔊 The Cocktail Party Analogy

Aliasing is like two people talking at the same time at a cocktail party. In Resolution IV, the loud speakers (main effects) are in separate rooms — you hear them clearly. But the quieter speakers (two-factor interactions) are paired up in the same room. You can hear that *someone* is talking, but can't always tell who. A foldover experiment is like asking one of them to step outside so you can hear the other clearly.

Foldover: Resolving Aliases

If a fractional factorial reveals ambiguous results (you can't tell if the effect comes from AB or CD), a **foldover** resolves the ambiguity. A foldover reverses the signs of one or more columns and runs the resulting design as a supplementary experiment.

Full Foldover

Reversing *all* column signs and combining with the original design doubles the runs but upgrades the resolution by one level. A Resolution III design becomes Resolution IV. This is the safest option when you're unsure which aliases are causing problems.

Single-Factor Foldover

Reversing just one column (say, factor A) de-aliases all effects involving A. This is more efficient when you suspect a specific factor is involved in the ambiguous interaction.

Worked Example: Full Foldover of a 2^{4-1}

If our 2^{4-1} analysis showed a large "AB+CD" effect and we can't tell which pair is responsible, we run a full foldover (reverse all signs):

Original (D = ABC)

Alias: AB + CD

8 runs, Resolution IV

Foldover (D = -ABC)

Alias: AB - CD

8 additional runs, Resolution IV

Combining the two half-fractions:

$$AB = \tfrac{1}{2}[(AB+CD)_{original} + (AB-CD)_{foldover}]$$
$$CD = \tfrac{1}{2}[(AB+CD)_{original} - (AB-CD)_{foldover}]$$

The combined 16-run design is equivalent to a full 2^4 factorial. All aliases are resolved. The cost: you've doubled your runs from 8 to 16, but you only do this when the initial analysis reveals ambiguity worth resolving. In many cases, the initial 8-run fraction is sufficient.

Augmentation in the DOE Helper Tool

The `doe augment` command automates design augmentation. It supports fold-over, star points (to upgrade a factorial to CCD), and center points (to detect curvature and estimate pure error):

```
# Fold-over to de-alias effects
doe augment --config config.json --type fold_over

# Add star points for RSM
doe augment --config config.json --type star_points

# Add center points to detect curvature
doe augment --config config.json --type center_points
```

The augmented script only runs the new experiments — it skips already-completed runs automatically.

Choosing the Right Fraction

The DOE Helper Tool selects minimum-aberration generators automatically, but here's the decision logic:

FACTORS	RECOMMENDED DESIGN	RUNS	RESOLUTION
2–4	Full factorial	4–16	—
5	2^{5-1}	16	V
6	2^{6-1}	32	VI
7	2^{7-2}	32	IV
8–11	Plackett-Burman or 2^{k-p}	12–32	III–IV
12+	Plackett-Burman screening first	varies	III

Fractional Factorial Workflow

A typical fractional factorial study follows this pattern:

1. **Start with the right resolution.** For initial screening, Resolution III or IV. For characterization, Resolution IV+.
2. **Run the fraction.** Analyze main effects and aliased interaction estimates.
3. **Decide next steps:**
   - If main effects are clear and no interactions look significant → **done**, proceed to optimization.
   - If some aliased interaction pairs look significant → **foldover** to resolve them.
   - If many factors are insignificant → **drop them** and run a full factorial or RSM on the survivors.

Fractional factorial with the DOE Helper Tool

```
# Config: 5 factors, fractional factorial (2^(5-1) = 16 runs instead of 32)
$ doe info --config config.json
  Operation: fractional_factorial
  Factors: 5 (all 2-level)
  Base runs: 16 (Resolution V)

# Generate and run
$ doe generate --config config.json --seed 42
$ bash run_experiments.sh

# Analyze: main effects, interactions, Pareto chart
$ doe analyze --config config.json

# Result: factors A and C dominate. Move to CCD with just those two.
```

The Power of Sequential Experimentation

Don't try to learn everything in one experiment. Run a fraction, learn what you can, then decide what to do next. This iterative approach — called **sequential experimentation** — is more efficient than any single design, no matter how clever. Each round of experiments informs the design of the next round.

Chapter 5

Screening Designs

> *When you have many factors and limited budget, Plackett-Burman designs let you screen them all with remarkably few runs.*

Plackett-Burman Designs

Plackett-Burman designs are 2-level screening designs that require only N+1 runs for N factors (rounded up to a multiple of 4). They estimate main effects only — all interactions are confounded.

8 runs	12 runs	20 runs
for 7 factors	for 11 factors	for 19 factors

How Plackett-Burman Works

PB designs are constructed from special cyclic generator sequences. For N=12 runs, the first row of the design is a specific sequence of + and - signs. Each subsequent row is formed by shifting the previous row one position to the right (wrapping around). The last row is all minus signs.

The resulting design is orthogonal for main effects but has complex confounding patterns for interactions. Unlike fractional factorials, where aliases follow a clean algebraic structure, PB confounding is "partially confounded" — each two-factor interaction is spread across multiple main effect estimates.

PB Confounding Is Messy

In a fractional factorial, you know exactly which effects are aliased (e.g., AB is aliased with CD). In a Plackett-Burman design, each two-factor interaction contributes partially to *multiple* main effect columns. This means a large interaction could inflate the apparent effect of an unrelated factor. This is acceptable for screening (where you only need to identify candidates), but not for characterization.

Example: 12-Run Plackett-Burman Matrix

A PB-12 design for up to 11 factors. Here we use 7 real factors (A–G) and 4 dummy columns (d1–d4):

RUN	A	B	C	D	E	F	G	d1	d2	d3	d4
1	+	+	-	+	+	+	-	-	-	+	-
2	-	+	+	-	+	+	+	-	-	-	+
3	+	-	+	+	-	+	+	+	-	-	-
4	-	+	-	+	+	-	+	+	+	-	-
5	-	-	+	-	+	+	-	+	+	+	-
6	-	-	-	+	-	+	+	-	+	+	+
7	+	-	-	-	+	-	+	+	-	+	+
8	+	+	-	-	-	+	-	+	+	-	+
9	+	+	+	-	-	-	+	-	+	+	-
10	-	+	+	+	-	-	-	+	-	+	+
11	+	-	+	+	+	-	-	-	+	-	+
12	-	-	-	-	-	-	-	-	-	-	-

The grey columns (d1–d4) are dummy factors. Their estimated effects measure only noise, providing a reference for judging which real factors are significant.

The Screening-to-Optimization Pipeline

The Two-Stage Strategy

Stage 1 (Screen): Use Plackett-Burman to test all candidates. Identify the 2–3 factors with the largest effects.
Stage 2 (Optimize): Run a full factorial, CCD, or Box-Behnken on just those important factors to find the true optimum.
Result: ~25 total runs instead of hundreds, with a rigorous path from "which factors matter?" to "what's the best setting?"

Limitations

Plackett-Burman cannot estimate interaction effects. If you suspect strong interactions, use a fractional factorial (Resolution IV+) instead. Also, PB requires exactly 2 levels per factor.

The Three-Stage Pipeline in Detail

Stage 1: Screen	→	Stage 2: Characterize	→	Stage 3: Confirm
Plackett-Burman		Factorial or RSM		Verification runs
7+ factors → 8–12 runs		2–3 factors → 15–20 runs		1 setting → 3–5 runs
Find the vital few		Model the response surface		Validate the optimum

Total cost: ~25–35 runs to go from "I have 7+ factors and no idea which matter" to "I have a validated optimum setting." Compare this to a full 2^7 factorial alone (128 runs) or grid search (thousands of runs).

🎯 The Funnel Analogy

Screening is like casting a wide net. You start with many candidates and quickly eliminate the irrelevant ones. Characterization narrows the focus to the survivors. Confirmation is the final check. Each stage is cheaper than the last because you're studying fewer factors at higher resolution. This funnel-shaped approach is the most efficient way to optimize any process.

Worked Example: HPC Cluster Screening

An HPC team suspects that 7 parameters affect job throughput: MPI ranks per node, OpenMP threads, memory allocation policy, I/O scheduler, network buffer size, file striping count, and checkpoint interval. Testing all 128 combinations of a full 2^7 factorial is impractical — each run takes 20 minutes on a shared cluster.

A Plackett-Burman design requires only **8 runs** (2 hours of wall-clock time). The Pareto chart from the analysis reveals that MPI ranks per node and network buffer size together account for 72% of the throughput variation. The other five factors are noise.

Full Factorial

128 runs × 20 min = **42 hours**

All 127 effects estimable

Only 2 effects actually matter

Plackett-Burman + Follow-up

8 screening + 4 factorial = **4 hours**

Identifies the 2 important factors

Full interaction detail on the vital few

Interpreting the Screening Results

The analysis output from the HPC screening might look like this:

```
doe analyze output

=== Main Effects: throughput_jobs_hr ===
Factor                    Effect    Std Error   % Contribution
---------------------------------------------------------------
mpi_ranks_per_node         45.2         3.1            42.8%
network_buffer_size        30.8         3.1            29.2%
io_scheduler                8.4         3.1             8.0%
file_striping_count         7.1         3.1             6.7%
openmp_threads              5.9         3.1             5.6%
memory_policy               4.2         3.1             4.0%
checkpoint_interval         3.9         3.1             3.7%
```

The first two factors (MPI ranks and network buffer) together account for 72% of the variation. The remaining five factors each contribute less than 10%. Using the dummy-factor approach, suppose the average dummy effect is 3.0. Only MPI ranks (45.2) and network buffer (30.8) clearly exceed the 2× threshold (6.0), confirming them as the active factors.

The Decision Point

After screening, you face a choice:
• **Drop the 5 inactive factors** and run a full factorial or CCD on MPI ranks + network buffer (as few as 9–13 runs).
• **Keep io_scheduler** (borderline at 8.0%) as a third factor in the follow-up (15–20 runs).
• **Fix all 5 inactive factors** at their cheaper/simpler levels (e.g., default memory policy, shortest checkpoint interval).
The screening data gives you the information to make this decision rationally.

Dummy Factors and Error Estimation

If your Plackett-Burman design has more columns than real factors, the leftover columns become **dummy factors** — factors that don't correspond to any real variable. Their estimated effects should be zero (they're just noise), so they provide an estimate of experimental error.

Using Dummy Factors

In a 12-run PB design with 7 real factors, you have 4 dummy columns. The effects of these dummies estimate the noise floor. Any real factor whose effect exceeds 2–3 times the average dummy effect is likely significant. This is a practical alternative to formal hypothesis testing when you have no replicates.

Practical Rules for Dummy Factor Analysis

CRITERION	RULE	RATIONALE
Definitely active	|Effect| > 3 × avg dummy effect	Very unlikely to be noise
Possibly active	2–3 × avg dummy effect	Worth investigating in follow-up
Inactive	|Effect| < 2 × avg dummy effect	Consistent with noise

If you have no dummy columns (e.g., a PB-8 with exactly 7 real factors), use the normal probability plot instead. The tool generates Pareto charts that visually separate the large effects from the noise floor — the steep initial bars are the active factors, and the remaining short bars are noise.

Definitive Screening Designs

Definitive Screening Designs (DSDs), introduced by Jones and Nachtsheim (2011), are a modern alternative to Plackett-Burman for continuous factors. They require only 2k+1 runs (for k factors), but can estimate:

- All main effects
- All quadratic effects
- Some two-factor interactions (when factors are dropped from the model)

DSDs use three levels per factor (-1, 0, +1), so they can detect curvature that PB designs miss. The trade-off is slightly more runs and the requirement that factors be continuous (not categorical).

When to Choose DSDs Over PB

Use DSDs when all factors are continuous and you suspect curvature may be important. Use Plackett-Burman when you have categorical factors, when minimizing runs is critical, or when you're confident that a linear model is sufficient for screening purposes.

Comparing Screening Designs

DESIGN	RUNS (FOR 7 FACTORS)	MAIN EFFECTS	QUADRATIC	INTERACTIONS	FACTOR TYPES
Plackett-Burman	8	Yes	No	Confounded	Any (2-level)
Fractional Factorial (Res III)	8	Yes (aliased w/ 2FI)	No	Aliased	Any (2-level)
Fractional Factorial (Res IV)	16	Yes (clear)	No	Aliased in pairs	Any (2-level)
Definitive Screening	15	Yes	Yes	Some	Continuous only

Quick Decision Guide

- **Many factors, all 2-level, budget-constrained:** Plackett-Burman
- **Need clean main effects with known alias structure:** Fractional factorial
- **All continuous factors, suspect curvature:** Definitive Screening Design
- **Fewer than 5 factors, each run is cheap:** Skip screening, go straight to full factorial or RSM

Screening Workflow with DOE Helper

Complete screening workflow

```
# Config: 7 factors, Plackett-Burman (8 runs)
$ doe info --config screening_config.json
  Operation: plackett_burman
  Factors: 7 (2-level each)
  Base runs: 8

# Generate, run, analyze
$ doe generate --config screening_config.json --seed 42
$ bash run_experiments.sh
$ doe analyze --config screening_config.json

# Result: factors A and C are active. Create a new config for optimization.
$ doe info --config optimization_config.json
  Operation: central_composite
  Factors: 2 (continuous)
  Base runs: 13

# Run the optimization study
$ doe generate --config optimization_config.json --seed 42
$ bash run_experiments.sh
$ doe analyze --config optimization_config.json
$ doe optimize --config optimization_config.json
```

Chapter 6

Response Surface Designs

> *When screening has identified the key factors, response surface designs add star points and center replicates to fit quadratic models and find the true optimum.*

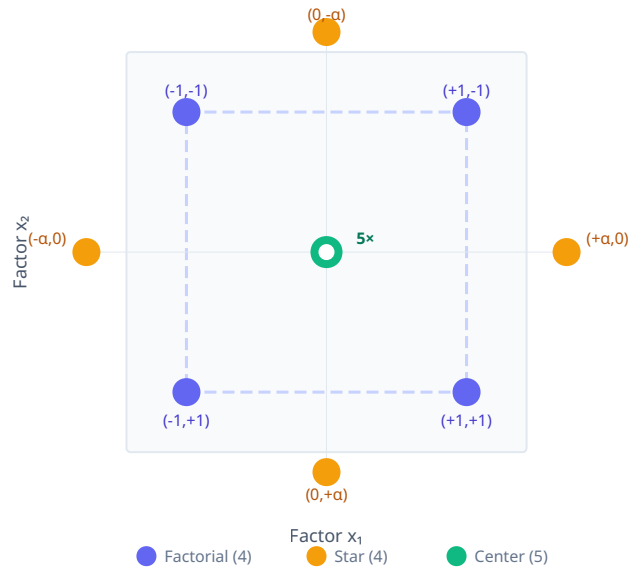
Central Composite Design (CCD)

A CCD consists of three types of experimental points, each serving a distinct purpose:

1. **Factorial points** (2^k): the corners of the design cube. These estimate linear effects and interactions — the same information a full factorial provides.
2. **Star (axial) points** (2k): points along each axis at distance α from center. These provide the additional information needed to estimate quadratic (curvature) terms.
3. **Center replicates** (c): repeated runs at the center, providing a direct estimate of experimental error and enabling the lack-of-fit test.

Here's what a CCD looks like for 2 factors:

Central Composite Design — 2 Factors



The factorial points (purple) form a square at the corners. The star points (amber) extend outward along each axis. The center points (green) are stacked at the middle. Together, these 13 runs provide enough information to fit all 6 terms of a two-factor quadratic model (intercept, two linear, one interaction, two quadratic).

FACTORS	FACTORIAL	STAR	CENTER	TOTAL	MODEL TERMS FITTED
2	4	4	5	**13**	6
3	8	6	6	**20**	10
4	16	8	7	**31**	15
5	32	10	8	**50**	21

Why CCD Works: Building On What You Have

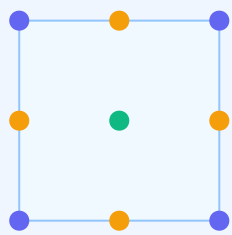
A major practical advantage of CCD is that you can build it **sequentially**. If you've already run a 2^k factorial with center points, you don't need to start over — just add the star points. This saves the factorial runs and lets you upgrade from a linear model to a quadratic model incrementally.

The Alpha Parameter in CCD

The star-point distance α controls where the axial points sit relative to the factorial cube. This single parameter determines the shape and statistical properties of the entire design:

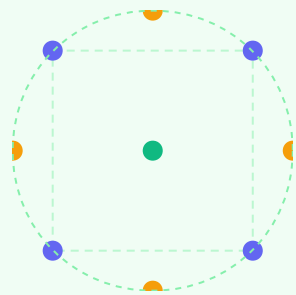
CHOICE	A VALUE (2 FACTORS)	PROPERTY	WHEN TO USE
Face-centered	1.00	Star points on cube faces; only 3 levels needed per factor	Factors with only 3 feasible levels; simplest to implement
Rotatable	1.414 (= $2^{k/4}$)	Prediction variance depends only on distance from center, not direction	General-purpose; best when you don't know which direction matters most
Spherical	1.414 (= √k)	All points lie on a sphere of equal radius	When the factor space has natural circular/spherical boundaries

Face-Centered (α = 1)



Star points sit on the cube faces. Only 3 levels per factor. Simplest.

Rotatable (α = 1.41)



Star points extend beyond the cube. 5 levels per factor. Equal prediction precision in all directions.

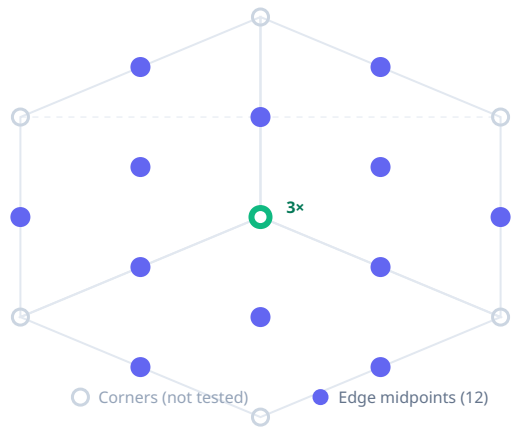
Practical Consideration: Factor Ranges

With a rotatable CCD (α > 1), the star points lie outside the factorial range. For 2 factors, α = 1.414 means the star points are 41% beyond the original range. Make sure these extreme values are physically feasible and safe *before* generating the design. If they aren't, use face-centered (α = 1).

Box-Behnken Design

Box-Behnken is an alternative RSM design that **never tests the corner conditions**. Every run has at most two factors at their extreme levels — the remaining factors stay at center. This makes Box-Behnken ideal when the corners represent dangerous, expensive, or physically impossible combinations.

Box-Behnken Design — 3 Factors (15 runs)



In a 3-factor Box-Behnken, the 12 design points sit at the midpoints of the cube's edges, plus 3 center replicates. No run has all three factors at their extremes simultaneously. Compare this to a CCD, which tests every corner:

PROPERTY	CCD (3 FACTORS)	BOX-BEHNKEN (3 FACTORS)
Total runs	20 (8+6+6)	15 (12+3)
Corner conditions tested?	Yes (all 8 corners)	**No**
Levels per factor	5 (rotatable) or 3 (face-centered)	3
Can be built sequentially?	Yes (add stars to factorial)	No (requires all points)
Quadratic model fittable?	Yes	Yes

CCD vs. Box-Behnken: Decision Guide

Use CCD when: Corner conditions are safe and feasible, you want to build on an existing factorial design, or you want the flexibility of choosing α.

Use Box-Behnken when: Corner conditions are dangerous or impossible (e.g., max temperature + max pressure + max flow rate in a reactor), when you want fewer total runs for 3+ factors, or when only 3 levels per factor are practical.

When Corner Conditions Are Dangerous

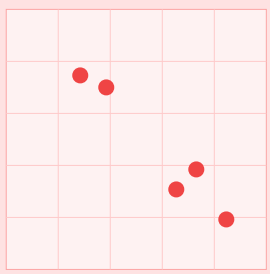
In a chemical reactor, running at maximum temperature, maximum pressure, and maximum catalyst simultaneously could cause a runaway reaction. In a server cluster, setting all resource parameters to maximum simultaneously could cause cascading failures. Box-Behnken avoids these extremes by design — no run ever has more than two factors at their limits.

Latin Hypercube Sampling

Latin Hypercube Sampling (LHS) is fundamentally different from factorial and RSM designs. Instead of testing specific combinations at discrete levels, LHS generates a **space-filling** set of points across the full continuous factor range.

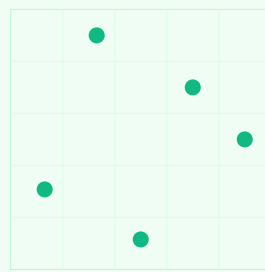
For N samples and k factors, LHS divides each factor's range into N equal intervals and places exactly one sample in each interval for every factor:

Random Sampling (Clumps)



Points cluster, leaving large unexplored gaps.

Latin Hypercube (Space-Filling)



Exactly one point per row and column. Uniform coverage guaranteed.

♟ Think of It Like Sudoku

In a Latin Hypercube, each row and column of the sampling grid has exactly one point — like placing non-attacking rooks on a chessboard. This guarantees that no region of the factor space is left unexplored, unlike pure random sampling which can leave large gaps.

When to Use LHS vs. Factorial Designs

SITUATION	USE LHS	USE FACTORIAL/CCD
Goal	Explore a large space; build surrogate models	Estimate specific effects; find precise optimum
Factors	Many continuous factors (5+)	Few factors (2–5)
Levels	Continuous range; many unique values	Discrete: 2–5 levels per factor
Model type	Gaussian process, neural net, or any ML model	Linear/quadratic polynomial
Statistical tests	None built-in; need bootstrap or cross-validation	ANOVA, t-tests, confidence intervals

LHS Cannot Estimate Effects Directly

Unlike factorial designs, LHS does not produce orthogonal columns. You cannot compute clean main effects or interaction effects from LHS data. LHS is best paired with flexible modeling techniques (Gaussian processes, random forests) that don't require orthogonality. If you need traditional DOE analysis (Pareto charts, effect estimates, ANOVA), use factorial or RSM designs.

Quick Reference: Choosing a Design

This is the most important table in the book. Use it to select your design based on what you need to learn:

QUESTION YOU NEED TO ANSWER	DESIGN	TYPICAL RUNS
Which of these 10+ factors matter?	**Plackett-Burman**	8–20
Screen factors + detect curvature?	**Definitive Screening**	2k+1
Main effects + some interactions? (5–8 factors)	**Fractional Factorial**	16–32
All effects, no compromises? (2–4 factors)	**Full Factorial**	4–16
Robust design with signal-to-noise ratios?	**Taguchi**	4–27
Find the quadratic optimum? Corners safe?	**CCD**	13–50
Find the quadratic optimum? Corners dangerous?	**Box-Behnken**	13–46
Custom run count, maximum information?	**D-Optimal**	Any
Uniform coverage of a large continuous space?	**Latin Hypercube**	10–100+
Formulation where components sum to 1?	**Mixture (Simplex)**	Depends on components

The Design Selection Flowchart

Step 1: Is this a mixture/formulation problem? If yes, use Mixture Simplex designs.
Step 2: How many factors? If > 6, start with screening (PB or Definitive Screening). DSD is preferred when you suspect curvature — it detects it in the same experiment.
Step 3: Do you need interactions? If no, PB is sufficient. If yes, use fractional or full factorial.
Step 4: Do you suspect curvature? Check center points. If center-point mean differs from factorial mean, upgrade to CCD or Box-Behnken.
Step 5: Are corner conditions feasible? If yes, CCD. If no, Box-Behnken.
Step 6: Is the factor space continuous with many dimensions? Consider LHS for initial exploration.
Step 7: Need a specific number of runs? Use D-Optimal to maximize information per run.
Step 8: Need robust design against noise factors? Use Taguchi orthogonal arrays.
Step 9: Need to extend an existing design? Use `doe augment` with fold-over, star points, or center points.

Worked Example: Chemical Process Optimization

A chemical engineer has identified two critical factors from a screening study: reaction temperature (150–200°C) and catalyst concentration (1–5%). They choose a rotatable CCD (α = 1.414) with 5 center replicates, giving 13 runs total.

RUN	TYPE	X_1 (CODED)	X_2 (CODED)	TEMP (°C)	CATALYST (%)	YIELD (%)
1	Factorial	-1	-1	150	1	62
2	Factorial	+1	-1	200	1	74
3	Factorial	-1	+1	150	5	78
4	Factorial	+1	+1	200	5	85
5	Star	-1.414	0	139.6	3	66
6	Star	+1.414	0	210.4	3	79
7	Star	0	-1.414	175	0.17	58
8	Star	0	+1.414	175	5.83	82
9–13	Center	0	0	175	3	88, 87, 89, 86, 88

Step 1: Check for Curvature

The average of the 4 factorial points is (62+74+78+85)/4 = **74.75%**. The average of the 5 center points is (88+87+89+86+88)/5 = **87.6%**. The center-point average is 12.85% higher than the factorial average — strong evidence of quadratic curvature. The optimum lies inside the design space, not at the corners.

The Curvature Test Is Your First Check

If center-point mean ≈ factorial mean, the surface is approximately flat (linear) in this region. Use steepest ascent to move toward the optimum. If center-point mean is significantly higher (or lower), curvature is present and the CCD data can map it. In our example, the 12.85% gap confirms strong curvature.

Step 2: Interpret the Fitted Model

The full quadratic model fitted to all 13 runs yields the prediction equation (see Chapter 8 for complete RSM analysis). The fitted model finds the predicted optimum at approximately **180°C and 3.5% catalyst**, with a predicted yield of **89.2%**.

Step 3: What the Star Points Reveal

The star points at extreme temperature (runs 5–6) give yields of 66% and 79% — both well below the center points. Similarly, extreme catalyst (runs 7–8) give 58% and 82%. This confirms that both factors have strong downward curvature at the extremes — there really is a peak in the middle.

What If Center Points Weren't Higher?

If the center yields had been ~75% (close to the factorial average), the linear model would be adequate and the CCD star points wouldn't add much value. You'd use the factorial data alone and apply steepest ascent to move toward the optimum. Always check center points first before committing to the full quadratic analysis.

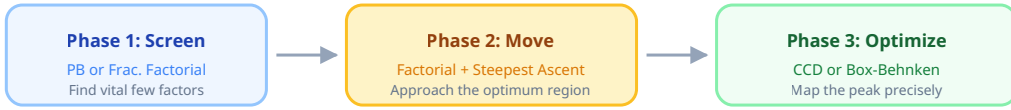
Sequential Experimentation

RSM designs are most powerful when used **sequentially**, in stages that build on each other. This avoids the common mistake of running an expensive, comprehensive experiment in a region far from the optimum.

The Three-Phase RSM Strategy

1. **Phase 1 — Screen:** Run a Plackett-Burman or fractional factorial to identify the 2–4 important factors from the original 8–15 candidates. (Chapter 5)
2. **Phase 2 — First-order model + steepest ascent:** Run a 2^k factorial with center points on the important factors. If the surface is approximately linear (center ≈ factorial average), use steepest ascent to move toward the optimum. Stop when the response levels off or starts declining.
3. **Phase 3 — Second-order model:** Center a CCD or Box-Behnken around the best point from Phase 2. Fit the full quadratic model. Locate the precise optimum. Run confirmation experiments.

Sequential RSM Strategy



⛰ The Hill-Climbing Analogy

Imagine finding the peak of a mountain in fog. **Phase 1** is looking around to see which directions slope upward (screening). **Phase 2** is walking uphill in the steepest direction (steepest ascent). **Phase 3** is stopping when the ground levels off and mapping the summit precisely (CCD). This iterative approach is far more efficient than trying to map the entire mountain at once.

How Many Total Runs?

A typical three-phase RSM study on 10 candidate factors:
Phase 1: 12-run Plackett-Burman identifies 3 important factors.
Phase 2: 8-run factorial + 3 center points + 5 steepest ascent runs = 16 runs.
Phase 3: 20-run CCD + 3 confirmation runs = 23 runs.
Total: ~51 runs from "which factors matter?" to "here's the validated optimum." A full factorial on 10 factors at 2 levels would need 1,024 runs — and still wouldn't find the quadratic optimum.

Chapter 7

Analyzing Results

> *Once you've run your experiments, the analysis phase turns raw data into actionable knowledge. The DOE Helper Tool automates this entire process.*

The Pareto Chart

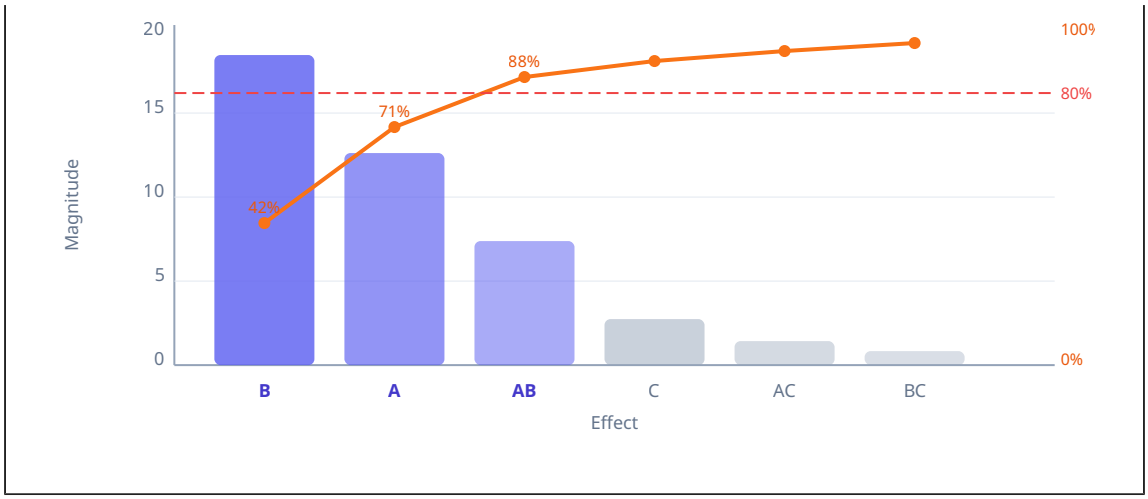
A Pareto chart ranks all effects by magnitude and shows their cumulative contribution. It is the single most important chart in DOE analysis — one glance tells you which factors matter and which don't.

Consider a 2^3 factorial on web server tuning with factors Thread Count (A), Cache Size (B), and Connection Timeout (C). The analysis produces these effect magnitudes:

EFFECT	MAGNITUDE	% CONTRIBUTION	CUMULATIVE %
B (Cache Size)	18.4	42.1%	42.1%
A (Threads)	12.6	28.8%	70.9%
AB	7.3	16.7%	87.6%
C (Timeout)	2.8	6.4%	94.0%
AC	1.4	3.2%	97.2%
BC	0.8	1.8%	99.0%
ABC	0.4	1.0%	100%

The Pareto chart renders this as descending bars with a cumulative line:

Pareto Chart of Effects — Server Throughput



How to Interpret This Pareto Chart

The bars (purple) are sorted tallest to shortest. Cache Size (B) dominates, followed by Threads (A). Together with their interaction (AB), these three effects account for 88% of all variation — past the 80% threshold (dashed red line).

The orange cumulative line rises steeply at first, then flattens. Once it crosses 80%, the remaining effects are noise. Factors C, AC, and BC (the grey bars) contribute less than 12% combined — you can fix Connection Timeout at any convenient value.

Action: The AB interaction is large (16.7%), which means Threads and Cache Size don't act independently. You must optimize them *together*, not separately. This is exactly the insight OVAT would miss.

Main Effects Plots

A main effects plot shows the average response at each level of each factor. It answers the question: *"When I change this factor from low to high, what happens to the response on average?"*

Using the same web server example, suppose the mean throughput at each factor level is:

FACTOR	LOW LEVEL MEAN	HIGH LEVEL MEAN	EFFECT
A (Threads): 10 vs 100	660 req/s	920 req/s	+260
B (Cache): 64MB vs 512MB	580 req/s	1000 req/s	+420
C (Timeout): 5s vs 30s	770 req/s	810 req/s	+40

Main Effects Plot — Server Throughput (req/s)

How to Interpret Main Effects Plots

Steep line = large effect. Factor B (Cache Size) has the steepest slope — it produces a 420 req/s improvement when going from low to high. This is the most impactful knob to turn.

Moderate line = moderate effect. Factor A (Threads) also matters, with a 260 req/s gain. Worth optimizing, especially given its interaction with B.

Flat line = negligible effect. Factor C (Timeout) barely changes the response (40 req/s). Set it to whatever is operationally convenient — it won't affect throughput meaningfully.

Direction matters. All three lines slope upward, meaning higher levels improve throughput. If a line sloped downward, the low level would be better for that factor.

Main Effects Plots Can Be Misleading

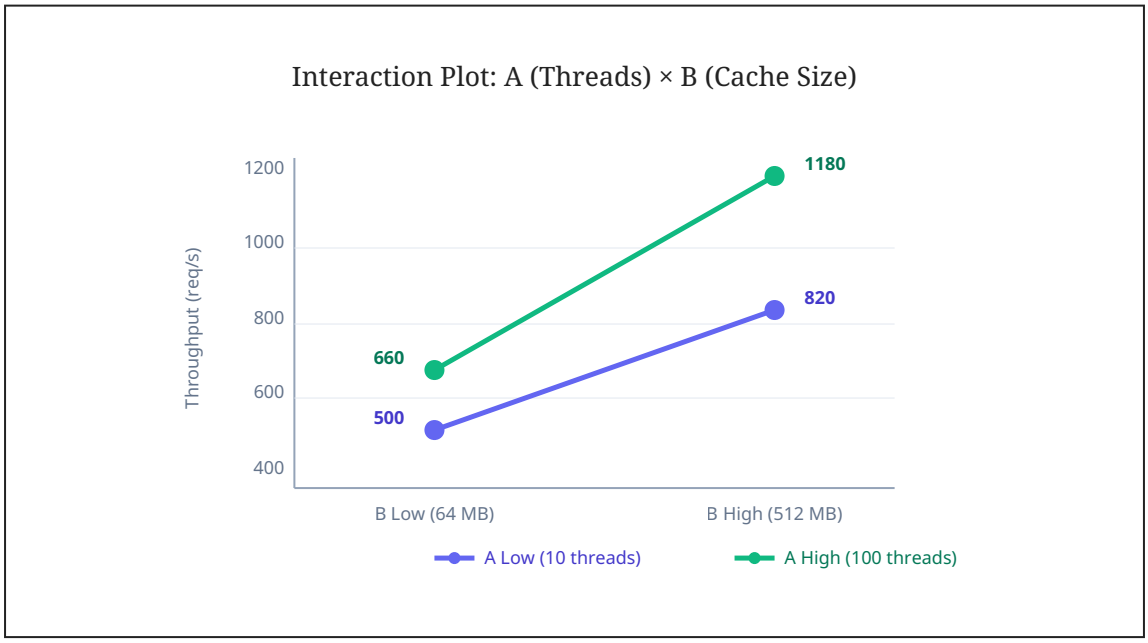
A main effects plot shows *averages* across all levels of all other factors. If a strong interaction exists (like AB here), the average can mask what's really happening. Factor A might have a huge effect when B is high but almost no effect when B is low. Always check the interaction plot before drawing conclusions from main effects alone.

Interaction Plots

Interaction plots are the most important diagnostic in DOE. They show whether the effect of one factor *depends on the level of another factor*. This is information that OVAT experiments can never provide.

Continuing our server example, here are the mean throughput values for all four AB combinations:

	B LOW (64MB)	B HIGH (512MB)
A Low (10 threads)	500	820
A High (100 threads)	660	1180



How to Interpret Interaction Plots

Non-parallel lines = interaction. The two lines in this plot diverge — the green line (A High) rises much more steeply than the purple line (A Low). This means the benefit of more cache is *amplified* when you also have more threads. This is a **synergistic interaction**.

Read the gap between lines. At B Low, the gap between A Low (500) and A High (660) is 160 req/s. At B High, the gap widens to 360 req/s (820 vs 1180). The effect of threads *more than doubles* when cache is large.

The practical implication: If you could only change one factor, you'd pick Cache Size (B). But the interaction tells you that changing *both* together yields 1180 — far more than the sum of their individual effects (160 + 320 = 480 above baseline of 500, giving 980). The extra 200 req/s is pure interaction benefit.

For comparison, here's what different interaction patterns look like and what they mean:

Parallel Lines = No Interaction

Both lines have the same slope. The effect of A is the same regardless of B's level. Each factor acts independently. You can optimize them separately.

Diverging Lines = Synergistic

Lines spread apart. The combined effect is *greater* than the sum of individual effects. Both factors should be set high (or both low) together. This is the pattern in our example.

Converging Lines = Antagonistic

Lines come together. One factor cancels part of the other's effect. The combined improvement is less than expected. You may need to choose which factor to prioritize.

Crossed Lines = Strong Interaction

Lines cross over. The *direction* of one factor's effect reverses depending on the other. The main effect average is meaningless here — only the interaction tells the real story. This is the most dangerous pattern to miss.

Statistical Significance

The tool computes 95% confidence intervals for each effect. A confidence interval tells you the range within which the true effect likely falls, accounting for experimental noise. If the interval does not include zero, the effect is statistically significant — meaning it's unlikely to be just noise.

Here's how the confidence intervals look for our server tuning example (with 2 replicates, SE = 2.1):

EFFECT	ESTIMATE	95% CI	SIGNIFICANT?
B (Cache)	18.4	[14.0, 22.8]	**Yes**
A (Threads)	12.6	[8.2, 17.0]	**Yes**
AB	7.3	[2.9, 11.7]	**Yes**
C (Timeout)	2.8	[-1.6, 7.2]	No (includes 0)
AC	1.4	[-3.0, 5.8]	No
BC	0.8	[-3.6, 5.2]	No

The confidence interval for C (Timeout) spans from -1.6 to 7.2 — it includes zero, so we cannot confidently say Timeout has any real effect. The positive estimate of 2.8 could easily be noise. By contrast, Cache Size's interval [14.0, 22.8] is entirely above zero and far from it, so we're confident it's a real and large effect.

Both Statistically & Practically Significant

Effect: 12.5 ± 3.2

CI: [9.3, 15.7]

Large effect, CI excludes zero. **Act on this.**

Statistically but Not Practically Significant

Effect: 0.3 ± 0.1

CI: [0.2, 0.4]

CI excludes zero, but the effect is tiny. **Probably ignore.**

Practical vs. Statistical Significance

Statistical significance depends on sample size. With enough data, even trivially small effects become "statistically significant." Always ask: *"Is this effect large enough to matter in my application?"* A 0.1% throughput improvement is statistically real but not worth engineering effort. A 20% improvement is worth chasing even if the p-value is marginal. Define your minimum meaningful effect size *before* the experiment.

Residual Analysis

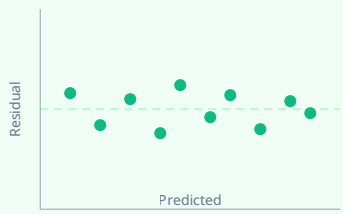
After fitting a model, always check the **residuals** (observed minus predicted values). The model assumes residuals are random, independent, and normally distributed with constant variance. If these assumptions fail, your effect estimates and confidence intervals may be unreliable.

There are four key residual plots to check:

1. Residuals vs. Predicted Values

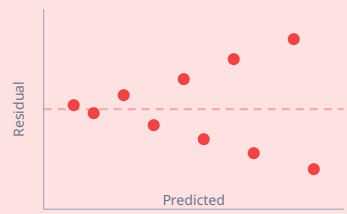
Plot residuals on the Y-axis against predicted (fitted) values on the X-axis. You want a random scatter with no pattern:

Good: Random Scatter



Points scattered randomly around zero. Model assumptions hold.

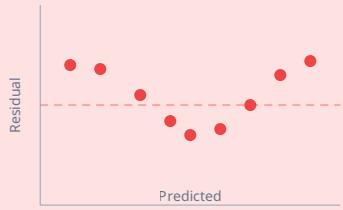
Bad: Funnel Shape



Variance increases with predicted value. Try a log transform.

Bad: Curved Pattern

Bad: Time Trend



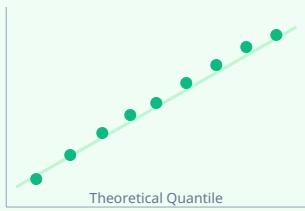
U-shape means missing quadratic terms. Add squared terms or use CCD.

Residuals drift over time. Use blocking or randomize run order.

2. Normal Probability Plot of Residuals

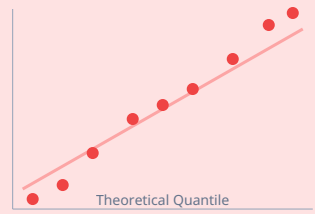
Plot residuals against expected normal quantiles. If the model is adequate and the data is well-behaved, points fall along a straight diagonal line. Deviations indicate non-normality:

Good: Points on the Line



Residuals are approximately normal. Analysis is valid.

Bad: S-Curve (Heavy Tails)



S-shape = heavy-tailed distribution. Check for outliers.

Residual Analysis Summary

What you want: Random scatter in residuals vs. predicted, straight line in normal plot, no trends over time, no outliers.

What to do if it fails:
• **Funnel shape:** Transform the response (log Y, √Y, or 1/Y). Log is the most common choice.
• **Curved pattern:** Your linear model is missing curvature. Add center points to detect it, or upgrade to CCD/Box-Behnken to fit quadratic terms.
• **Time trend:** Something drifted during the experiment (temperature, tool wear, cache warming). Block by time or re-randomize and repeat.
• **Outlier:** Investigate the specific run. Was there a measurement error? Equipment issue? If you can identify the cause, remove it and note why. If not, keep it — the model should be robust to one outlier.

Multi-Response Analysis

Real experiments often have multiple responses. A database tuning study might measure throughput, latency, and CPU usage. Different responses may have conflicting optima — the settings that maximize throughput might also maximize CPU usage.

Consider a server optimization with two responses:

SETTING	THROUGHPUT (REQ/S)	CPU USAGE (%)
A Low, B Low	500	35%
A High, B Low	660	52%
A Low, B High	820	48%
A High, B High	1180	88%

The best throughput (1180) comes with 88% CPU usage — dangerously close to saturation. You need to find the sweet spot.

Strategies for Multiple Responses

1. Overlay plots: Plot contour maps of each response on the same axes. The feasible region is where all responses are within acceptable bounds. Visually identify the best operating point in the overlap zone.

2. Desirability functions: Convert each response to a 0–1 desirability score (0 = unacceptable, 1 = ideal) and maximize the geometric mean. This balances competing objectives automatically. The `optimize` command supports this approach.

3. Prioritize and constrain: Optimize the primary response (throughput) while constraining secondary responses to acceptable ranges (CPU < 70%). This is often the most practical approach — it frames the problem as *"What's the best I can do without breaking anything?"*

Effect Size and Power

Before running an experiment, consider: how large an effect do you need to detect? The **power** of a design is the probability of detecting a real effect of a given size. Higher power requires more replicates.

Power depends on four things:

FACTOR	EFFECT ON POWER	YOU CONTROL IT?
Effect size (signal)	Larger effect = easier to detect	Indirectly (widen factor ranges)
Noise (std deviation)	More noise = harder to detect	Partially (better measurement, blocking)
Sample size (replicates)	More runs = more power	Yes (set `block_count`)
Significance level (α)	Stricter threshold = less power	Yes (usually 0.05)

As a rough guide: a 2^k design with 2 replicates can detect effects about 2–3 times the noise standard deviation with 80% power. If your effects are likely smaller than that, you need more replicates or a larger design.

The DOE Helper Tool computes power for each factor using the non-central F distribution:

```
# Check power before running the experiment
doe power --config config.json --sigma 2.0 --delta 5.0

# Or estimate sigma from existing results
doe power --config config.json --delta 5.0 --results-dir results/
```

If power is below 0.80 for important factors, increase `block_count` in your config to add replicates, or widen your factor ranges to increase the expected effect size.

> 🔍 **The Signal-to-Noise Analogy**
>
> Detecting an effect in a noisy experiment is like hearing a conversation in a loud room. A large effect (loud voice) is easy to detect even with few replicates. A small effect (whisper) requires either a quiet room (low noise) or many listeners (many replicates) to be heard reliably.

Confirmation Runs

After identifying the best settings, always run **confirmation experiments** at those settings. This is the final validation step before deploying to production. Compare the confirmation results to the model's prediction:

METRIC	MODEL PREDICTION	CONFIRMATION RESULT	VERDICT
Throughput	1150 ± 85 req/s	1120, 1165, 1138	**Within prediction interval**
Latency	12.3 ± 2.1 ms	11.8, 13.1, 12.4	**Within prediction interval**

If the confirmation results fall within the model's prediction interval, the model is validated and you can proceed with confidence. If they don't:

- **Results consistently higher or lower:** The model has a bias. Check for a lurking variable that changed between the experiment and confirmation.
- **Results much more variable:** The experimental conditions weren't fully reproduced. Document what differed.

- **Results completely off:** The model is inadequate. Consider additional terms, a wider design, or re-examining your assumptions.

Don't Skip Confirmation

A model is a simplification of reality. No matter how good the R^2, always confirm with real data before committing to production settings. Three confirmation runs is a reasonable minimum. If the stakes are high (production database, safety-critical system), run five or more.

Managing Real-World Experiments

Not all experiments are automated scripts. Physical experiments — lab tests, field measurements, manufacturing trials — require a different workflow. The DOE tool provides three commands specifically for this:

The Manual Experiment Toolkit

`export-worksheet` generates a printable run sheet (CSV or Markdown) with all factor settings and blank columns for recording responses. `status` shows a progress bar and tells you which run to do next. `record` lets you enter results interactively with validation. Together, they replace the test script in the automated workflow.

A typical real-world experiment proceeds in stages, often over days or weeks:

1. **Before the first run:** Generate the design, then export a worksheet. Print it or open it in a spreadsheet.
2. **During the experiment:** After each run, enter results with `record`. Use `status` to see what's left.
3. **Mid-experiment analysis:** Use `analyze --partial` to get early insights. This is especially valuable for expensive experiments — if one factor clearly dominates after half the runs, you can make informed decisions about how to proceed.
4. **After completion:** Run the full analysis, optimization, and report pipeline.

```
Complete manual workflow

# Print a worksheet for the lab
$ doe export-worksheet --config config.json --format csv --output runs.csv

# After each experiment, record the result
$ doe record --config config.json --run 1
$ doe record --config config.json --run 2

# Check progress
$ doe status --config config.json

# Peek at partial results
$ doe analyze --config config.json --partial

# When all runs are done
$ doe analyze --config config.json
$ doe optimize --config config.json
$ doe report --config config.json --output report.html
```

Partial Analysis Caveat

Partial results give directional insights but not definitive conclusions. Effect estimates from incomplete designs may be biased, especially if the missing runs are not randomly distributed. Always confirm findings with the complete dataset.

Putting It All Together: The Analysis Workflow

Here is the recommended sequence for analyzing any DOE result:

1. **Check the Pareto chart.** Which factors are in the vital few? Are there any surprises (unexpected interactions)?
2. **Examine main effects plots.** What direction does each important factor push the response? How large are the effects in practical units?
3. **Study interaction plots.** Do the important factors interact? If lines cross, the main effect averages are misleading — optimize the combination, not the individual factors.
4. **Verify with confidence intervals.** Are the important effects statistically significant? Are the unimportant ones truly negligible?

5. **Check residual plots.** Is the model adequate? Are the assumptions met? If not, transform the data or upgrade the design.
6. **Run confirmation experiments.** Does reality match the model's prediction at the recommended settings?

The Most Common Mistake in DOE Analysis

Looking only at main effects and ignoring interactions. In our server example, if you only looked at main effects, you'd set both A and B to high and expect 660 + 420 = 1080 req/s above the baseline. But the interaction gives you an extra 100 req/s (1180 total), and more importantly, if you had *missed* the interaction and only changed one factor, you'd capture far less improvement than possible. Always check the interaction plots.

Chapter 8

Response Surface Methodology

> *RSM moves beyond "which factors matter" to "what is the optimal setting?" by fitting a mathematical model to the experimental data.*

From Effects to Models

In Chapters 3–7 you learned to estimate *effects* — the change in response when a factor moves from low to high. Effects tell you the **direction** of improvement. But they don't tell you what happens at intermediate settings, or exactly where the optimum lies. That's where RSM comes in.

RSM fits a mathematical model to the experimental data, turning your discrete measurements into a continuous **prediction equation**. For two factors the full quadratic model is:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_{12} x_1 x_2 + \beta_{11} x_1^2 + \beta_{22} x_2^2$$

Each β term captures a specific type of behaviour:

TERM	WHAT IT MODELS	EXAMPLE
β_0	Overall average response at the center of the design	88.5% yield at 175°C, 3% catalyst
$\beta_1 x_1$	Linear effect of temperature	Higher temperature increases yield by 6.2 per coded unit
$\beta_2 x_2$	Linear effect of catalyst	More catalyst increases yield by 8.1 per coded unit
$\beta_{12} x_1 x_2$	Interaction — synergy or antagonism	Temperature + catalyst together boost yield by an extra 2.3
$\beta_{11} x_1^2$	Curvature in the temperature direction	Yield peaks and then drops at very high temp ($\beta_{11} < 0$)
$\beta_{22} x_2^2$	Curvature in the catalyst direction	Yield peaks at moderate catalyst ($\beta_{22} < 0$)

📐 **The Terrain Map Analogy**

Think of RSM as building a topographic map from a handful of elevation measurements. The linear terms tell you which direction is uphill. The quadratic terms tell you whether the terrain curves — whether there's a peak, a valley, or a saddle. The interaction term tells you whether the slope in one direction changes as you move in the other direction. Once you have the map, you can find the peak without visiting every single point.

First-Order vs. Second-Order Models

Not every RSM study needs quadratic terms. The two model orders serve different purposes:

First-Order (Linear + Interactions)

$\hat{y} = \beta_0 + \Sigma\beta_i x_i + \Sigma\beta_{ij} x_i x_j$

Use when: You're far from the optimum and need to find the direction of steepest ascent. Requires only factorial + center points.

Second-Order (Full Quadratic)

$\hat{y} = \beta_0 + \Sigma\beta_i x_i + \Sigma\beta_{ij} x_i x_j + \Sigma\beta_{ii} x_i^2$

Use when: You're near the optimum and need to map the peak precisely. Requires CCD or Box-Behnken.

Worked Example: Reactor Yield Optimization

A chemical process team has screened 7 factors down to 2: temperature (x_1, 150–200°C) and catalyst concentration (x_2, 1–5%). They run a CCD with 5 center replicates (13 runs total). After fitting the full quadratic model, the `optimize` command reports:

```
RSM Output

# Fitted Model (coded units):
ŷ = 87.6 + 6.2·x₁ + 8.1·x₂ + 2.3·x₁x₂ - 5.8·x₁² - 4.1·x₂²

# Model Fit:
R²        = 0.964
R²(adj)   = 0.938
Lack-of-fit p-value = 0.31  (not significant → model is adequate)

# Stationary Point (coded):
x₁ = +0.48,  x₂ = +0.85

# Stationary Point (natural units):
Temperature = 187°C,  Catalyst = 3.7%

# Predicted optimum:
ŷ = 92.1% yield
```

Let's walk through how to interpret every part of this output.

Reading the Coefficients

COEFFICIENT	VALUE	INTERPRETATION
$\beta_0 = 87.6$	Intercept	At the center point (175°C, 3%), predicted yield is 87.6%
$\beta_1 = +6.2$	Temperature (linear)	Moving from center to high temperature adds 6.2% yield
$\beta_2 = +8.1$	Catalyst (linear)	Catalyst has a larger linear effect than temperature
$\beta_{12} = +2.3$	Interaction	Mild synergy — both high together gives a bonus 2.3%
$\beta_{11} = -5.8$	Temperature (quadratic)	**Negative** → yield curves *downward* at extreme temperatures. There's a peak.
$\beta_{22} = -4.1$	Catalyst (quadratic)	**Negative** → yield also curves downward with extreme catalyst. There's a peak.

Both quadratic coefficients are negative, which means the response surface is dome-shaped — there's a true maximum inside the experimental region. This is the ideal outcome for RSM.

Model Adequacy

Before trusting any prediction, you must verify that the model actually fits the data. The tool reports several diagnostics automatically:

- **R^2 and Adjusted R^2** — Overall model fit.
- **PRESS statistic and Predicted R^2** — Leave-one-out cross-validation measure that penalizes influential observations. A large gap between R^2 and Predicted R^2 indicates overfitting.
- **ANOVA table** — F-tests and p-values for each term, lack-of-fit test, and Lenth's PSE for unreplicated designs.
- **Diagnostic plots** — Residuals vs. fitted values (check for patterns), normal probability plot of residuals (check for normality), residuals vs. run order (check for drift), and predicted vs. actual (check for bias). These are generated automatically by `doe analyze`.

R^2 and Adjusted R^2

R^2	INTERPRETATION	ACTION
> 0.9	Excellent fit — model explains most variation	Proceed with optimization
0.7 – 0.9	Good fit — useful for optimization	Proceed, but widen confirmation runs
0.5 – 0.7	Moderate — model is missing something	Check for missing quadratic terms or transform the response
< 0.5	Poor fit — the model doesn't describe the data	Redesign the experiment: wider range, more factors, different model

In our reactor example, $R^2 = 0.964$ means the model explains 96.4% of the variation in yield. Adjusted $R^2 = 0.938$ penalizes for the number of terms — still high, confirming that the terms in the model are justified, not just overfitting noise.

R^2 vs. Adjusted R^2: When to Worry

If $R^2 = 0.95$ but adjusted $R^2 = 0.72$, you have too many terms for the amount of data. Remove non-significant terms (usually the interaction first, then the smaller quadratic) and refit. A gap greater than ~0.10 between R^2 and adjusted R^2 is a red flag for overfitting.

ANOVA Table

The analysis of variance breaks the model into individual terms and tests each one:

SOURCE	SUM OF SQUARES	DF	F-VALUE	P-VALUE	SIGNIFICANT?
x_1 (Temp, linear)	307.4	1	38.2	0.0003	**Yes**
x_2 (Cat, linear)	524.9	1	65.3	<0.0001	**Yes**
x_1x_2	21.2	1	2.6	0.15	No
x_1^2	198.3	1	24.7	0.0011	**Yes**
x_2^2	99.1	1	12.3	0.0079	**Yes**
Lack of fit	15.4	3	1.5	0.31	Not significant (good)
Pure error	13.6	4	—	—	—

How to Read the ANOVA Table

Significant terms ($p < 0.05$): Both linear effects and both quadratic effects are significant — these terms belong in the model.

Non-significant terms: The interaction x_1x_2 ($p = 0.15$) is borderline. You could remove it to simplify the model, but keeping a mild interaction term usually doesn't hurt prediction accuracy.

Lack of fit ($p = 0.31$): This is *not* significant, which is what you want. It means the quadratic model fits the data adequately — there's no evidence of a pattern that the model misses. If this were significant ($p < 0.05$), you'd need a more complex model.

Finding the Optimum

The `optimize` command reports three things: the **best observed run** (the highest actual measurement), the **predicted optimum at observed points** from the RSM model, and the **true surface optimum** found by numerically optimizing the fitted polynomial using L-BFGS-B with multiple random restarts.

Best Observed Run

Temperature: 175°C

Catalyst: 3%

Yield: **89.0%**

This is a real measurement from a center-point replicate. It's trustworthy but might not be the true optimum — you only tested discrete points.

Predicted Optimum (RSM)

Temperature: **187°C**

Catalyst: **3.7%**

Yield: **92.1%** ± 2.4

This is the model's prediction at an untested point. It's the peak of the fitted surface. Must be validated with confirmation runs.

The predicted optimum is 3.1% better than the best observed run, and it occurs at a setting nobody tested directly. This is the power of RSM — interpolation between experimental points to find a better answer than any individual run.

Method of Steepest Ascent

If your first-order model suggests the optimum is far outside your experimental region (the contour lines are nearly parallel, pointing off the edge of the plot), use the **method of steepest ascent** before committing to a full CCD.

1. Fit a first-order (linear) model to your factorial data.
2. Compute the gradient direction: the path of steepest increase in the predicted response.
3. Run a few experiments along that path, stepping outward from the center, until the response stops improving.
4. Center a new CCD around the best point found on the path.

The DOE Helper Tool automates steps 1–2 with the `--steepest` flag:

```
doe optimize --config config.json --steepest
```

This generates a table of recommended follow-up experiment points along the gradient direction, with predicted response values at each step.

⛰ Hill-Climbing in Fog

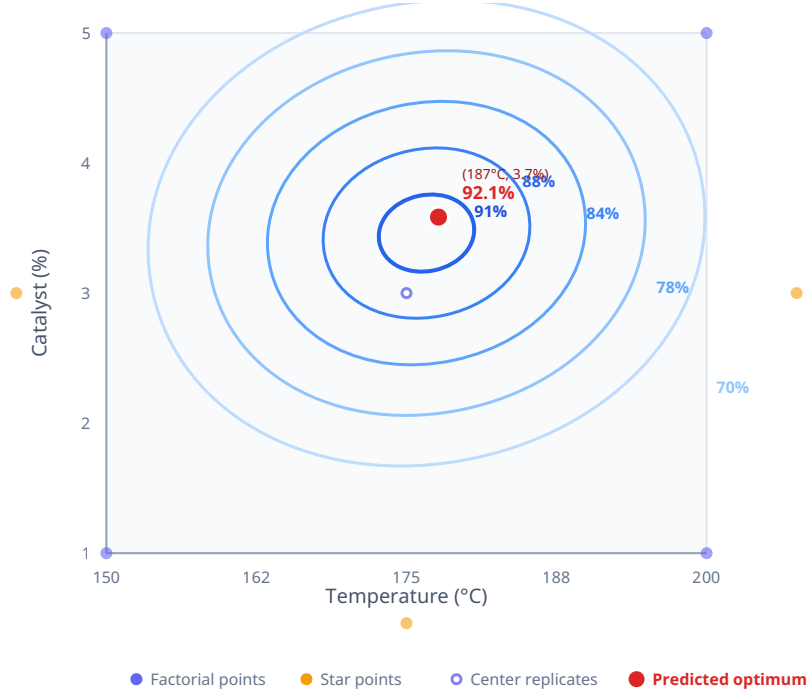
You're on a mountainside in fog. You can't see the summit, but you can feel the slope under your feet. Steepest ascent says: walk uphill in the steepest direction. When the ground levels off (response stops improving), stop and survey the area carefully (run a CCD). This two-phase approach avoids wasting a full CCD in a region far from the peak.

Contour Plots and Surface Plots

The fitted RSM model can be visualized as a **contour plot** (2D topographic map) or a **surface plot** (3D wireframe). Contour plots are more practical for decision-making because you can read exact coordinates from them.

Here is the contour plot for our reactor example, showing predicted yield as a function of temperature and catalyst concentration:

Contour Plot — Predicted Yield (%) vs. Temperature × Catalyst



How to Read This Contour Plot

Each contour line connects points with the same predicted yield. The labels (70%, 78%, 84%, 88%, 91%) show the yield at each contour level. Moving inward toward the center of the concentric ellipses means higher yield.

The red dot marks the predicted optimum (92.1% at 187°C, 3.7% catalyst). It lies inside the innermost contour, slightly above and to the right of the design center.

The ellipse tilt reveals the interaction. The contours are tilted about 12° from horizontal, meaning temperature and catalyst have a mild synergy (the AB interaction). If there were no interaction, the ellipses would be aligned with the axes.

Contour spacing tells you sensitivity. Contours are more tightly packed in the temperature direction (left-right), meaning yield is more sensitive to temperature near the edges of the region. Along the catalyst axis (up-down), contours are more spread out, meaning yield changes more gradually.

Here's what different contour patterns mean and how to act on them:

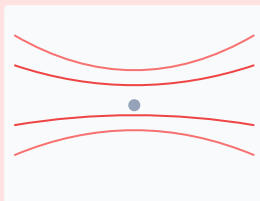
Concentric Ellipses: Clear Peak

Optimum is inside the region. Run confirmation at the predicted peak.

Parallel Lines: Ridge System

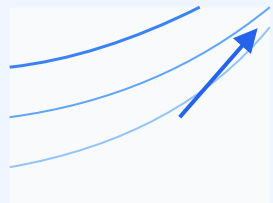
Only one factor matters here. The other has no curvature. Set the flat factor to any convenient value.

Saddle Point: No True Optimum



Eigenvalues have mixed signs. The best point is on the boundary. Use ridge analysis.

Open Contours: Optimum Outside



The optimum is beyond your tested range. Use steepest ascent to move toward it, then re-center a new CCD.

Canonical Analysis

Canonical analysis transforms the fitted model into a simpler form that reveals the nature of the response surface at its stationary point. The quadratic RSM model can be written in matrix form as:

$$\hat{y} = \hat{y}_s + \lambda_1 w_1^2 + \lambda_2 w_2^2$$

where $\hat{y}_s$ is the predicted response at the stationary point, λ_i are the eigenvalues of the quadratic coefficient matrix B, and w_i are the rotated coordinates. The original matrix form is:

$$\hat{y} = \beta_0 + x'b + x'Bx \quad \Rightarrow \quad x_s = -\tfrac{1}{2} B^{-1} b$$

The eigenvalues tell you everything about the shape of the surface at the stationary point:

EIGENVALUES	SURFACE SHAPE	WHAT IT LOOKS LIKE	ACTION
All negative	**Maximum** (hill-top)	A dome — response decreases in every direction from the peak	The stationary point is your optimum. Confirm it.
All positive	**Minimum** (valley)	A bowl — response increases in every direction from the low point	If minimizing, this is your optimum. If maximizing, the best point is on the boundary.
Mixed signs	**Saddle point**	A horse saddle — a peak in one direction but a valley in the other	No true optimum exists. Use ridge analysis to find the best point on the boundary.
Some near zero	**Ridge system**	A mountain ridge — flat along one direction, curved in another	Multiple near-optimal points exist. Pick the one that's most practical or robust.

For our reactor example:

Canonical Analysis Output

```
# Eigenvalues of B matrix:
λ₁ = -6.13   (temperature direction, rotated)
λ₂ = -3.77   (catalyst direction, rotated)

# Both negative → surface is a MAXIMUM (dome)
# Predicted response at stationary point: 92.1%
# Stationary point in coded units: x₁=+0.48, x₂=+0.85
# Stationary point in natural: Temp=187°C, Cat=3.7%
```

Both eigenvalues are negative, confirming a true maximum. The magnitude ratio $|\lambda_1/\lambda_2| = 1.63$ tells us the surface is about 63% more curved in the temperature direction than in the catalyst direction — meaning yield is more sensitive to temperature deviations from the optimum. This has practical implications: you should control temperature more tightly than catalyst concentration in production.

Eigenvalue Magnitudes Matter

The absolute magnitude of each eigenvalue tells you how sensitive the response is along that direction. Large $|\lambda|$ means steep curvature — small deviations from the optimum cause big response changes. Small $|\lambda|$ means a flat ridge — you have more tolerance. When setting process tolerances, allocate tighter control to factors aligned with the largest eigenvalue.

Ridge Analysis

In roughly 30–40% of real RSM studies, the stationary point falls **outside the experimental region** or is a saddle point. In either case, the stationary point is not directly useful. Ridge analysis solves this by finding the best predicted response at each distance from the center of the design.

Think of it as drawing concentric circles (or spheres) outward from the center and asking: *"What's the highest yield I can achieve on each circle?"*

🎯 **The Fenced Garden Analogy**

If the highest point on your property is actually outside the fence, ridge analysis finds the highest point *on* the fence — the best you can achieve within your constraints. You can then decide whether to extend the fence (expand the experimental region) in a follow-up study.

A ridge analysis trace for a saddle-point example might look like:

RADIUS (CODED)	x_1	x_2	PREDICTED RESPONSE
0.0	0.00	0.00	85.2
0.2	-0.08	+0.18	86.1
0.4	-0.16	+0.36	87.8
0.6	-0.22	+0.56	89.9
0.8	-0.27	+0.75	91.5
1.0	-0.30	+0.95	**92.4**
1.2	*-0.32*	*+1.14*	*92.8 (outside region)*

Don't Extrapolate Beyond the Design

The model is fitted to data within the experimental region (coded -1 to +1). Predictions at radius > 1.0 are extrapolations — the model has no data there and may be wildly wrong. If the ridge trace shows the response still climbing at radius = 1.0, that's a signal to run a follow-up experiment centered further along the ridge direction.

The Method of Steepest Ascent

When you're far from the optimum, steepest ascent is the most efficient way to move toward it. The idea is simple: use the first-order model's coefficients to determine the direction of fastest improvement, then run a few experiments along that path.

From a first-order model with coefficients $\beta_1 = 6.2$ (temperature) and $\beta_2 = 8.1$ (catalyst), the steepest ascent path moves in the direction proportional to these coefficients. In natural units, you step along:

STEP	TEMP (°C)	CATALYST (%)	OBSERVED YIELD (%)
Center	175	3.0	82.4
+1 step	183	3.6	86.1
+2 steps	191	4.2	88.7
+3 steps	199	4.8	**89.3 (best)**
+4 steps	207	5.4	87.0 (declining)

Yield peaked at step 3 and declined at step 4 — you've passed the optimum. Center a new CCD around (199°C, 4.8%) to map the peak precisely.

When to Use Steepest Ascent vs. Direct CCD

Use steepest ascent when: Your initial factorial or screening study shows mostly linear effects with little curvature (center points close to factorial average). You're probably far from the peak and need to move toward it.

Skip to CCD directly when: Center points are noticeably higher (or lower) than the factorial average — this signals curvature, meaning you're already near the peak. A CCD will map it directly.

Lack-of-Fit Testing

If you have replicated center points (and you should — always include 3–5), you can formally test whether the quadratic model fits the data. The lack-of-fit test compares two sources of variation:

- **Pure error:** Variation among replicates at the same settings (estimated from center points). This is "real" noise — the inherent variability of the process.
- **Lack of fit:** Variation between model predictions and observed means at each unique design point. This is "systematic" error — patterns the model fails to capture.

The F-test asks: *"Is the model's prediction error significantly larger than the pure experimental noise?"*

Lack-of-Fit NOT Significant ($p > 0.05$)

The model fits the data adequately. Prediction errors are no larger than random noise. Proceed with optimization using this model.

Our reactor example: $p = 0.31$. The quadratic model is sufficient.

Lack-of-Fit IS Significant ($p < 0.05$)

The model is missing something systematic. Predictions deviate from reality in a patterned way. Do **not** trust the predicted optimum.

Remedies: Transform Y (log, sqrt), add cubic terms, restrict the region, or try a different model.

Multi-Response Optimization with Desirability

Most real processes have multiple responses to optimize simultaneously. A reactor produces yield, purity, and cost. A server has throughput, latency, and CPU usage. The settings that maximize one response often worsen another.

The **desirability function** approach converts each response to a 0–1 scale (0 = unacceptable, 1 = ideal), then combines them into a single score:

$$D = (d_1^{w_1} \times d_2^{w_2} \times \ldots \times d_k^{w_k})^{1/\Sigma w}$$

where d_i is the individual desirability for response i, and w_i is its importance weight.

Desirability Functions by Goal

GOAL	D_I = 0 WHEN	D_I = 1 WHEN	EXAMPLE
Maximize	Response ≤ lower bound	Response ≥ target	Yield: d = 0 below 70%, d = 1 above 95%
Minimize	Response ≥ upper bound	Response ≤ target	Cost: d = 0 above $50, d = 1 below $20
Target	Response outside bounds	Response = target	pH: d = 1 at exactly 7.0, d = 0 outside 6.0–8.0

Continuing the reactor example with three responses:

RESPONSE	GOAL	LOWER	TARGET/UPPER	WEIGHT
Yield (%)	Maximize	70	95	3 (high priority)
Purity (%)	Maximize	90	99	2
Cost ($/kg)	Minimize	15	40	1 (low priority)

Interpreting the Overall Desirability

D = 1.0: All responses have hit their ideal targets simultaneously — rare in practice.
D > 0.8: Excellent compromise. All responses are close to their targets.
D = 0.5–0.8: Acceptable compromise. Some responses are trading off against others.
D < 0.5: At least one response is far from its target. Revisit the weights or accept that the goals conflict strongly.
D = 0: At least one response is completely unacceptable. The geometric mean makes D = 0 if *any* individual $d_i = 0$ — this is a safety feature that prevents accepting solutions where any single response is terrible.

Using Multi-Objective Optimization in the DOE Helper Tool

The `--multi` flag on `doe optimize` implements Derringer-Suich desirability automatically. It fits RSM models for each response, evaluates desirability across the factor space, and reports the best compromise.

Multi-objective optimization

```
# Basic multi-objective (equal weights, auto bounds)
$ doe optimize --config config.json --multi
```

To control the trade-off, add `weight` and `bounds` to responses in your config:

```
config.json — weighted responses

"responses": [
  {"name": "yield",  "optimize": "maximize", "weight": 3,   "bounds": [70, 95]
},
  {"name": "purity", "optimize": "maximize", "weight": 2,   "bounds": [90, 99]
},
  {"name": "cost",   "optimize": "minimize", "weight": 1,   "bounds": [15, 40]
}
]
```

- **weight** (default 1.0) — relative importance. A weight of 3 means "this response matters 3× more" in the geometric mean.
- **bounds** (optional) — [worst, best] values for desirability scaling. For "maximize," the first value is the worst acceptable (d=0) and the second is the ideal (d=1). For "minimize," the first is ideal and the second is the worst. If omitted, bounds are auto-computed from observed data.

The output includes:

- **Overall desirability D** — the weighted geometric mean across all responses
- **Per-response desirability** — how well each response meets its target
- **Recommended settings** — either from observed runs or RSM-predicted optima
- **Trade-off summary** — how much each response sacrifices compared to its individual best
- **Top 3 runs** — the best observed factor combinations ranked by D

Choosing Weights Carefully

Weights express relative priority, not absolute importance. If you're unsure, start with equal weights (all 1.0) and examine the trade-offs. Then adjust: if the compromise sacrifices too much yield, increase yield's weight and re-run. Iterate until the trade-off is acceptable. Document your final weights — they encode a business decision about what matters most.

The Complete RSM Workflow

Bringing together everything in this chapter, here's the recommended sequence:

1. **Start with a first-order model.** Fit a factorial or PB design. Check center points for curvature. If center-point mean ≈ factorial mean, the surface is flat — use steepest ascent to move toward the optimum.
2. **Run steepest ascent.** Take 3–6 steps along the gradient. Stop when the response starts declining or leveling off.
3. **Center a CCD around the best point.** Run a CCD (or Box-Behnken if corners are infeasible). Fit the full quadratic model.
4. **Check model adequacy.** $R^2 > 0.9$? Lack of fit not significant? Residuals look random? If yes, proceed. If no, diagnose and fix.
5. **Interpret the contour plot.** Concentric ellipses with the optimum inside? Great. Saddle or open contours? Use ridge analysis or expand the region.
6. **Run canonical analysis.** Confirm the stationary point type. Check eigenvalue magnitudes for sensitivity information.
7. **For multiple responses, compute desirability.** Set targets and weights in your config, then run `doe optimize --config config.json --multi` to find the best compromise.
8. **Confirm.** Run 3–5 experiments at the predicted optimum. Compare to the prediction interval.

RSM Is Not Machine Learning

RSM fits simple polynomial models (usually 5–15 coefficients) to small datasets (13–50 runs). It works because the experimental region is small and the physics is smooth. Don't expect it to capture complex, highly nonlinear relationships across a vast design space — that's a different problem requiring different tools. RSM's strength is *precise optimization in a local region* with statistical guarantees, not global exploration.

Chapter 9

Practical Troubleshooting

> *Common issues and how to resolve them.*

"All my effects look the same size"

Likely cause: Factor ranges are too narrow, or the response is dominated by noise. **Fix:** Widen factor ranges (use more extreme levels) or add replicates to reduce noise.

"The confidence intervals are huge"

Likely cause: Too few replicates or high experimental noise. **Fix:** Increase `block_count` to add replicates, or reduce measurement variability.

"My optimizer says linear R^2 is low"

Likely cause: The true relationship is nonlinear (quadratic curvature). **Fix:** Switch from a screening design to CCD or Box-Behnken, which can fit quadratic terms.

"Some runs failed"

The generated runner scripts handle failures gracefully — they track failed runs and continue. After fixing the issue, re-run only the missing runs (the skip logic detects existing result files). Use `--partial` with analyze, optimize, or report to work with incomplete data:

```
Terminal

$ doe analyze --config config.json --partial
# Partial mode: analyzing 5/8 completed runs (missing: 3, 6, 7)
```

"I need more than 2 levels for screening"

Plackett-Burman and fractional factorial require exactly 2 levels. For multi-level screening, use a full factorial with a manageable subset of factors, or use Latin Hypercube for continuous factors.

"My model predicts values outside the physical range"

Likely cause: A linear or quadratic model extrapolating beyond the experimental region, or the response has a natural bound (e.g., yield can't exceed 100%). **Fix:** Apply a transformation (logit for proportions, log for positive-only values) before fitting the model, or constrain predictions to the feasible range during optimization.

"I have categorical and continuous factors mixed together"

This is common (e.g., algorithm type + buffer size). Use a full or fractional factorial design where categorical factors appear as coded levels. The tool handles mixed factor types — just set `type: "categorical"` or `type: "continuous"` in the config. Note: CCD and Box-Behnken require continuous factors, so you may need to run separate RSM studies for each level of the categorical factor.

"I don't have a test script — my experiments are physical/manual"

The tool fully supports manual experiments. Leave `test_script` empty in your config and use the manual workflow:

1. **Export a worksheet:** `doe export-worksheet --config config.json --format csv --output worksheet.csv`
2. **Check progress:** `doe status --config config.json`
3. **Record results:** `doe record --config config.json --run 1`
4. **Analyze incrementally:** `doe analyze --config config.json --partial`

The analysis pipeline doesn't care how results were produced — only that `run_N.json` files exist with the right response keys.

"My results aren't reproducible between days"

Likely cause: Day-to-day environmental variation (ambient temperature, system load, different hardware). **Fix:** Use blocking — set `block_count: 2` (or more) so each day is a block. Randomize run order within each block. The analysis separates block effects from treatment effects.

Common Mistakes Checklist

Before running your experiment, verify you haven't made these common errors:

MISTAKE	CONSEQUENCE	PREVENTION
Running factors one at a time	Missing interactions; biased effect estimates	Use a proper design (factorial, PB, etc.)
Not randomizing run order	Time trends confounded with factor effects	Always randomize; the tool does this by default
Factor ranges too narrow	All effects look insignificant	Use bold ranges (at least ±25% of current value)
Factor ranges too wide	Nonlinear behavior; model breaks down at extremes	Check feasibility of extreme combinations first
No replicates	No error estimate; can't assess significance	Add center points or use `block_count > 1`
Changing unmeasured variables between runs	Confounded effects	Document and control all known nuisance variables
Ignoring the response metric definition	Measuring the wrong thing	Define the response precisely before running (mean? median? p99?)

The DOE Workflow in Practice

Here is the complete workflow, from initial idea to validated optimum:

1. **Define the objective.** What are you optimizing? What constitutes success? Agree on quantifiable responses.
2. **List candidate factors.** Brainstorm with domain experts. Include factors you're *not sure* about — screening will sort them out.
3. **Set factor levels.** Choose bold but feasible ranges. The levels should be far enough apart that real effects are detectable.
4. **Choose the design.** Many factors? Screen first (PB). Few factors? Go straight to factorial or CCD.
5. **Randomize and run.** Follow the generated run order. For automated experiments, use the generated runner script. For manual experiments, use `export-worksheet` to print a run sheet, `status` to track progress, and `record` to enter results. Record everything, including conditions you didn't plan to vary.
6. **Analyze.** Use `--partial` to get early insights while experiments are still running. Compute effects, check residuals, identify the vital few factors.
7. **Follow up.** If screening, run a factorial or RSM on the important factors. If optimizing, run confirmation experiments.
8. **Confirm and implement.** Validate the optimum with independent runs. Then deploy.

Iteration Is Normal

DOE is rarely a one-shot process. Most real optimization studies require 2–3 rounds: screening, then characterization, then fine-tuning. Each round informs the next. The total cost is still far less than exhaustive search, and the conclusions are statistically defensible.